4 物理地址 ..... 2
4.1 从 CPU 角度看物理内存模型 ..... 2
4.1.1 FLATMEM 平坦内存模型 ..... 2
4.1.2 DISCONTIGMEM 非连续内存模型 ..... 3
4.1.3 SPARSEMEM 稀疏内存模型 ..... 5
4.2 物理内存热插拔 ..... 12
4.3 从 CPU 角度看物理内存架构 ..... 14
4.3.1 一致性内存访问 UMA 架构 ..... 14
4.3.2 非一致性内存访问 NUMA 架构 ..... 16
4.3.3 NUMA 的内存分配策略 ..... 16
4.3.4 内核如何管理 NUMA 节点 ..... 17
4.3.4.1 内核如何统一组织 NUMA 节点 ..... 17
4.3.4.2 NUMA 节点内的物理内存区域划分 ..... 20
4.3.4.3 NUMA 节点中的内存规整与回收 ..... 24
4.3.4.4 NUMA 节点的状态 node_states ..... 24
4.3.4.5 内核管理 NUMA 节点中的物理内存区域 ..... 25
4.4 物理内存区域中的预留内存 ..... 28
4.5 物理内存区域中的水位线 ..... 29
4.5.1 计算 min_free_kbytes ..... 32
4.5.2 计算物理内存区域的三条水位线 ..... 35
4.5.3 watermark_scale_factor 调整水位线的间距 ..... 37
4.6 物理内存区域中的冷热页 ..... 38
4.7 内核中描述物理内存页 ..... 40
4.8 匿名页的反向映射 ..... 43
4.8.1 反向映射中使用到的数据结构 ..... 44
4.8.2 匿名反向映射过程 ..... 46
4.8.2.1 sruct anon_vma 、 struct anon_vma_chain 和 struct page 三者之间建立联系 ..... 46
4.8.2.2 struct page 与进程虚拟内存空间 struct vm_area_struct 之间建立联系 ..... 48
4.8.2.3 sruct anon_vma 、 struct anon_vma_chain、 struct page、与 vm_area_struct 整体关系 ..... 49
4.9 内存页回收相关属性 ..... 56
4.10 物理内存页属性和状态的标志位 flag ..... 58
4.11 复合页 compound_page 相关属性 ..... 62
4.12 Slab 对象池相关属性 ..... 65

# 4 物理地址

## 4.1 从 CPU 角度看物理内存模型

内核是以页为基本单位对物理内存进行管理的，通过将物理内存划分为一页一页的内存块，每页大小为 4K。一页大小的内存块在内核中用 struct page 结构体来进行管理，struct page 中封装了每页内存块的状态信息，比如：组织结构，使用信息，统计信息，以及与其他结构的关联映射信息等。

为了快速索引到具体的物理内存页，内核为每个物理页 struct page 结构体定义了一个索引编号：**PFN（Page Frame Number）**。PFN 与 struct page 是一一对应的关系。

内核提供了两个宏来完成 PFN 与 物理页结构体 struct page 之间的相互转换。它们分别是 **page_to_pfn** 与 **pfn_to_page**。

内核中如何组织管理这些物理内存页 struct page 的方式我们称之为做**物理内存模型**，不同的物理内存模型，应对的场景以及 page_to_pfn 与 pfn_to_page 的计算逻辑都是不一样的。

### 4.1.1 FLATMEM 平坦内存模型

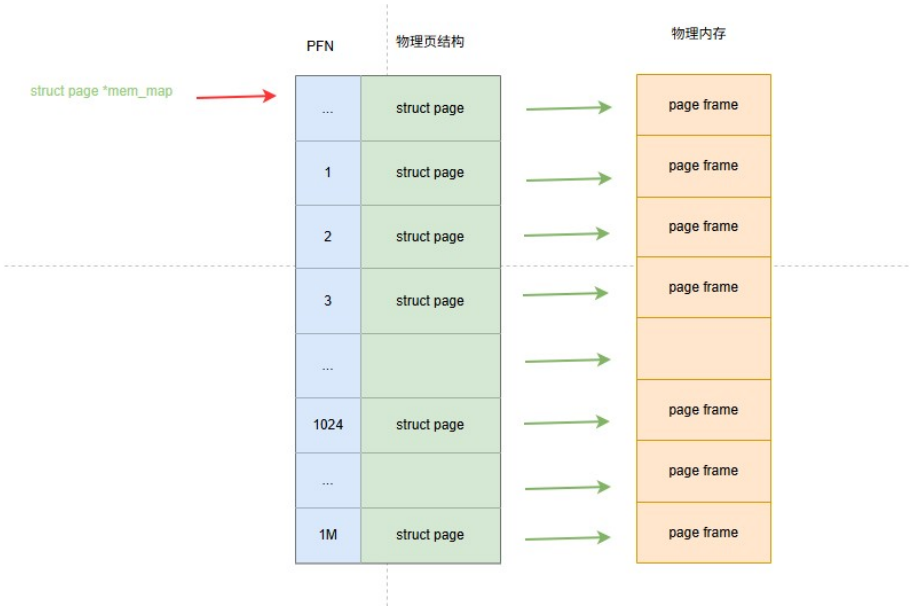
先把物理内存想象成一片地址连续的存储空间，在这一大片地址连续的内存空间中，内核将这块内存空间分为一页一页的内存块 struct page 。由于这块物理内存是连续的，物理地址也是连续的，划分出来的这一页一页的物理页必然也是连续的，并且每页的大小都是固定的，所以很容易想到用一个数组来组织这些连续的物理内存页 struct page 结构，其在数组中对应的下标即为 PFN 。这种内存模型就叫做**平坦内存模型 FLATMEM** 。

内核中使用了一个 node_mem_map 的全局数组用来组织所有划分出来的物理内存页。node_mem_map 全局数组的下标就是相应物理页对应的 PFN 。

在平坦内存模型下 ，page_to_pfn 与 pfn_to_page 的计算逻辑本质就是基于 node_mem_map 数组进行偏移操作。

```
#if defined(CONFIG_FLATMEM)

#define __pfn_to_page(pfn)  (mem_map + ((pfn) - ARCH_PFN_OFFSET))
#define __page_to_pfn(page) ((unsigned long)((page) - mem_map) + \
                ARCH_PFN_OFFSET)
#endif
```

其中，ARCH_PFN_OFFSET 是 PFN 的起始偏移量。



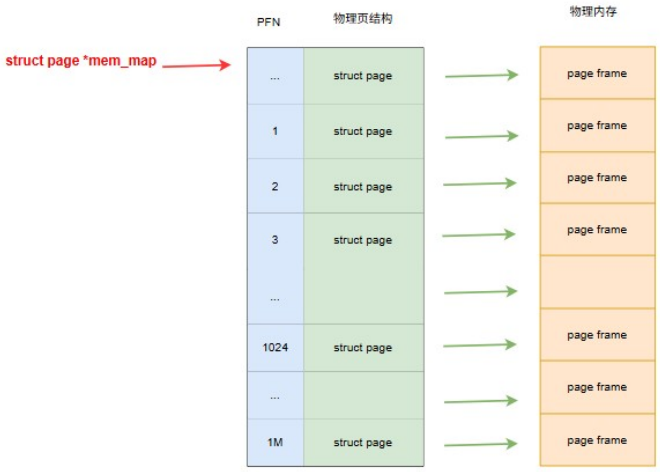

## 4.1.2 DISCONTIGMEM 非连续内存模型

FLATMEM 平坦内存模型只适合管理一整块连续的物理内存，而对于多块非连续的物理内存来说使用 FLATMEM 平坦内存模型进行管理则会造成很大的内存空间浪费。

因为 FLATMEM 平坦内存模型是利用 mem_map 这样一个全局数组来组织这些被划分出来的物理页 page 的，而对于物理内存存在大量不连续的内存地址区间这种情况时，这些不连续的内存地址区间就形成了内存空洞。

由于用于组织物理页的底层数据结构是 mem_map 数组，数组的特性又要求这些物理页是连续的，**所以只能为这些内存地址空洞也分配 struct page 结构用来填充数组使其连续。**

而每个 struct page 结构大部分情况下需要占用 40 字节（struct page 结构在不同场景下内存占用会有所不同），如果物理内存中存在的大块的地址空洞，那么为这些空洞而分配的 struct page 将会占用大量的内存空间，导致巨大的浪费。



为了组织和管理这些不连续的物理内存，内核引入了 DISCONTIGMEM 非连续内存模型，用来消除这些不连续的内存地址空洞对 mem_map 的空间浪费。

在 DISCONTIGMEM 非连续内存模型中，内核将物理内存从宏观上划分成了一个一个的节点 node （微观上还是一页一页的物理页），每个 node 节点管理一块连续的物理内存。这些连续的物理内存页均被划归到了对应的 node 节点中管理，就避免了内存空洞造成的空间浪费。

内核中使用 struct pglist_data 表示用于管理连续物理内存的 node 节点（内核假设 node 中的物理内存是连续的），既然每个 node 节点中的物理内存是连续的，于是在每个 node 节点中还是采用 FLATMEM 平坦内存模型的方式来组织管理物理内存页。每个 node 节点中包含一个 struct page *node_mem_map 数组，用来组织管理 node 中的连续物理内存页。

```
typedef struct pglist_data {
...... 省略 ......

#ifdef CONFIG_FLATMEM   /* means !SPARSEMEM */
    struct page *node_mem_map;
#ifdef CONFIG_PAGE_EXTENSION
    struct page_ext *node_page_ext;
#endif

...... 省略 ......
}
```

可以看出 DISCONTIGMEM 非连续内存模型其实就是 FLATMEM 平坦内存模型的一种扩展，在面对大块不连续的物理内存管理时，通过将每段连续的物理内存区间划归到 node 节点中进行管理，避免了为内存地址空洞分配 struct page 结构，从而节省了内存资源的开销。

由于引入了 node 节点这个概念，所以在 DISCONTIGMEM 非连续内存模型下 page_to_pfn 与 pfn_to_page 的计算逻辑就比 FLATMEM 内存模型下的计算逻辑多了一步定位 page 所在 node 的操作。

通过 **pfn_to_nid** 可以根据物理页的 PFN 定位到物理页所在 node。

通过 **page_to_nid** 可以根据物理页结构 struct page 定义到 page 所在 node。

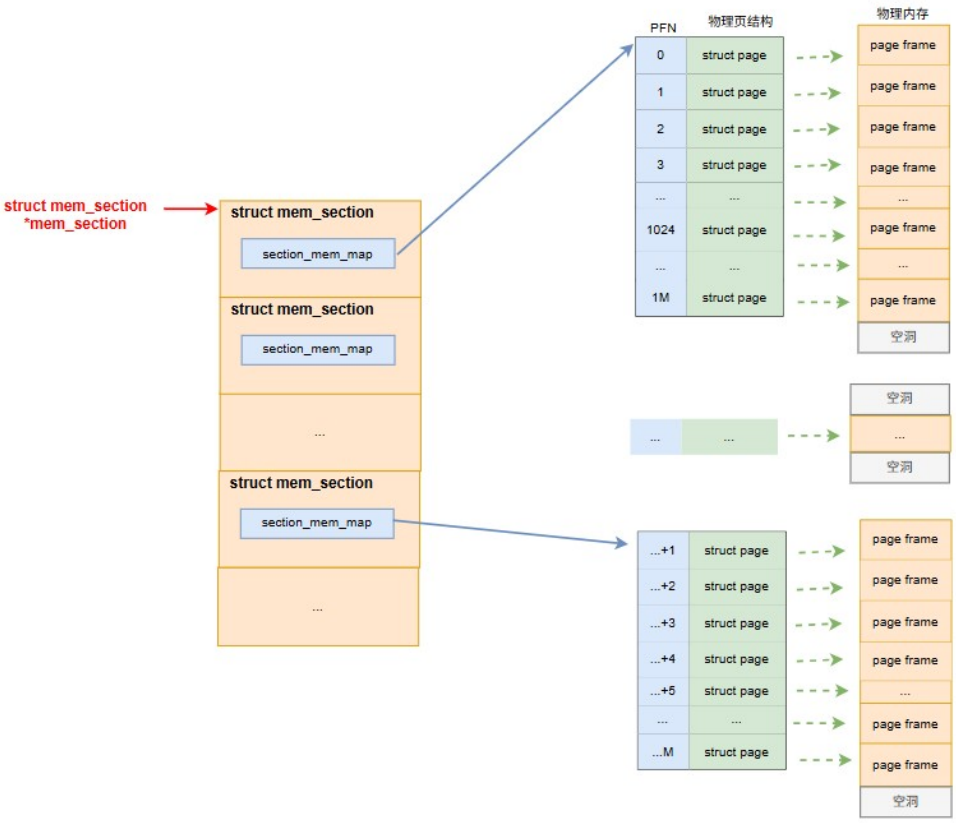
```
static inline int page_to_nid(const struct page *page)
{
    struct page *p = (struct page *)page;

    return (PF_POISONED_CHECK(p)->flags >> NODES_PGSHIFT) & NODES_MASK;
}

#define pfn_to_nid(pfn)                                \
({                                              \
    unsigned long __pfn_to_nid_pfn = (pfn);            \
    page_to_nid(pfn_to_page(__pfn_to_nid_pfn));        \
})
```

当定位到物理页 struct page 所在 node 之后，剩下的逻辑就和 FLATMEM 内存模型一模一样了。

DISCONTIGMEM 非连续内存模型 在 6.x 版本已经废弃了。

### 4.1.3 SPARSEMEM 稀疏内存模型

随着内存技术的发展，内核可以支持物理内存的热插拔了，这样一来物理内存的不连续就变为常态了，每个 node 中的物理内存也不一定都是连续的了。每个 node 中都有一套完整的内存管理系统，如果 node 数目多的话，那这个开销就大了，于是就有了对连续物理内存更细粒度的管理需求，为了能够更灵活地管理粒度更小的连续物理内存，就引入了人 SPARSEMEM 稀疏内存模型。




SPARSEMEM 稀疏内存模型的核心思想就是对粒度更小的连续内存块进行精细的管理，用于管理连续内存块的单元被称作 **section** 。**物理页大小为 4k 的情况下， section 的大小为 128M ，物理页大小为 16k 的情况下， section 的大小为 512M。**

MAX_PHYSMEM_BITS：架构支持的物理地址位宽（比如 x86_64 支持 48 位物理地址，就定义为 48），是「硬件决定的固定值」；

SECTION_SIZE_BITS：单个 section 的地址位宽（比如 28 对应 256MB），是「架构自定义的任意值」（只需平衡元数据开销和管理粒度）。

section 的最大数量用 NR_MEM_SECTIONS 表示，计算公式为：

NR_MEM_SECTIONS = （2 ^MAX_PHYSMEM_BITS）/（2 ^SECTION_SIZE_BITS）

补充：sparse_init() 函数的解析

单个 section 大小 = 2^SECTION_SIZE_BITS 字节，2^28 = 256MB

```
#if !defined(MAX_PHYSMEM_BITS) || !defined(SECTION_SIZE_BITS)
#define MAX_PHYSMEM_BITS    36
#define SECTION_SIZE_BITS   28
#endif
```

一个 NUMA 节点包含的 section 数量，本质是「节点的物理内存总大小 ÷ 单个

section 的大小」，公式如下：

节点的 section 数量 = (node_end_pfn - node_start_pfn) / SECTIONS_PER_PAGE? 不，核心公式是：

节点的 section 数量 = 节点覆盖的 PFN 范围 ÷ 单个 section 的 PFN 数量

```
/*
 * Allocate the accumulated non-linear sections, allocate a mem_map
 * for each and record the physical to section mapping.
 */
void __init sparse_init(void)
{
    //pnum_end: 当前 NUMA 节点的起始 section 编号
    //pnum_begin: 遍历过程中当前处理的结束 section 编号
    //map_count: 当前 NUMA 节点内连续的 section 数量
    unsigned long pnum_end, pnum_begin, map_count = 1;
    //nid_begin: 当前 NUMA 节点的 ID
    int nid_begin;
    //扫描内核启动早期的 memblock 内存块管理器,
    //标记哪些物理内存区域是「实际存在的（present）」,
    //为后续遍历「有效 section」做准备;
    memblocks_present();
    //遍历全局 mem_sections 数组，找到系统中第一个实际存在的 section 编号
    pnum_begin = first_present_section_nr();
    //__nr_to_section: 将「section 编号」转换为对应的
    //struct mem_section * 指针
    //sparse_early_nid: 从 section->section_mem_map 字段中解析出该
    //section 所属的 NUMA 节点 ID
    nid_begin = sparse_early_nid(__nr_to_section(pnum_begin));
    //初始化大页相关配置
    //初始化 pageblock_order（页块阶数）,
    //适配「可变大小的巨页（HUGETLB）」和内存迁移功能;
    set_pageblock_order();
    //核心循环（按节点分组连续 section）
    for_each_present_section_nr(pnum_begin + 1, pnum_end) {
    //遍历从 pnum_begin + 1 开始的所有实际存在的 section
    //每次循环，pnum_end 会被自动赋值为当前遍历到的 section 编号
    //只遍历 “有效 section”，跳过内存空洞对应的 section

    //获取当前 section 所属的节点 ID
        int nid = sparse_early_nid(__nr_to_section(pnum_end));
```

```
        if (nid == nid_begin) {
             //如果当前 section 和上一个 section 属于同一个节点，就累计计数
            map_count++;
            continue;
        }
        //如果当前 section 属于新节点，先初始化上一个节点的所有 section
        sparse_init_nid(nid_begin, pnum_begin, pnum_end, map_count);
       //更新上下文，切换到新节点
        nid_begin = nid;// 新节点 ID
        pnum_begin = pnum_end;// 新节点的起始 section 编号
        map_count = 1; // 新节点的 section 计数重置为 1
    }
    /* cover the last node */
    sparse_init_nid(nid_begin, pnum_begin, pnum_end, map_count);
    vmemmap_populate_print_last();
}
```

```
static inline int sparse_early_nid(struct mem_section *section)
{
    return (section->section_mem_map >> SECTION_NID_SHIFT);
}
```

```
/*
 * Initialize sparse on a specific node. The node spans [pnum_begin,
pnum_end)
 * And number of present sections in this node is map_count.
 */
static void __init sparse_init_nid(int nid, unsigned long pnum_begin,
                  unsigned long pnum_end,
                  unsigned long map_count)
{
    struct mem_section_usage *usage;
    unsigned long pnum;
    struct page *map;
    usage = sparse_early_usemaps_alloc_pgdat_section(NODE_DATA(nid),
            mem_section_usage_size() * map_count);
```

```
    if (!usage) {
        pr_err("%s: node[%d] usemap allocation failed", __func__, nid);
        goto failed;
    }
    sparse_buffer_init(map_count * section_map_size(), nid);
    for_each_present_section_nr(pnum_begin, pnum) {
        unsigned long pfn = section_nr_to_pfn(pnum);
        if (pnum >= pnum_end)
            break;
        map = __populate_section_memmap(pfn, PAGES_PER_SECTION,
                nid, NULL, NULL);
        if (!map) {
            pr_err("%s: node[%d] memory map backing failed. Some memory will
not be available.",
                   __func__, nid);
            pnum_begin = pnum;
            sparse_buffer_fini();
            goto failed;
        }
        check_usemap_section_nr(nid, usage);
        sparse_init_one_section(__nr_to_section(pnum), pnum, map, usage,
                SECTION_IS_EARLY);
        usage = (void *) usage + mem_section_usage_size();
    }
    sparse_buffer_fini();
    return;
failed:
    /* We failed to allocate, mark all the following pnums as not present
*/
    for_each_present_section_nr(pnum_begin, pnum) {
        struct mem_section *ms;
        if (pnum >= pnum_end)
            break;
        ms = __nr_to_section(pnum);
        ms->section_mem_map = 0;
    }
}
```

在内核中用 struct mem_section 结构体表示 SPARSEMEM 模型中的 section。

```
struct mem_section {
    ...
    unsigned long section_mem_map;
    ...
}
```

section 被用作管理小粒度的连续内存块，这些小的连续物理内存在 section 中也是通过数组的方式被组织管理，每个 struct mem_section 结构体中有一个 section_mem_map 指针用于指向 section 中管理连续内存的 page 数组。该字段是「多功能编码值」，并非单纯的指针，里面存储的内容包括「指针 + 节点 ID + 状态」。

SPARSEMEM 内存模型中的这些所有的 mem_section 会被存放在一个全局的数组中，并且每个 mem_section 都可以在系统运行时改变 offline / online （下线 / 上线）状态，以便支持内存的热插拔（hotplug）功能。

```
#ifdef CONFIG_SPARSEMEM_EXTREME
extern struct mem_section **mem_section;
#else
extern struct mem_section
mem_section[NR_SECTION_ROOTS][SECTIONS_PER_ROOT];
#endif
```

Linux 内核稀疏内存模型（SPARSEMEM）中 初始化单个 struct mem_section 的核心函数：

```
static void __meminit sparse_init_one_section(struct mem_section *ms,
        unsigned long pnum, struct page *mem_map,
        struct mem_section_usage *usage, unsigned long flags)
{
    //清空 section_mem_map 的低有效位
    ms->section_mem_map &= ~SECTION_MAP_MASK;
    //编码 struct page 数组（mem_map）地址
    ms->section_mem_map |= sparse_encode_mem_map(mem_map, pnum)
        | SECTION_HAS_MEM_MAP | flags;
    ms->usage = usage;
}
```

编码 struct page 数组（mem_map）地址。通过 “mem_map 指针与内存段起始 PFN 的偏移计算”，将 struct page 数组的地址编码为紧凑的 “偏移值”，后续可通过 “物理页 PFN 与该偏移值反向计算” 快速定位到对应的 struct page。

```
static unsigned long sparse_encode_mem_map(struct page *mem_map, unsigned
long pnum)
{
    unsigned long coded_mem_map =
        (unsigned long)(mem_map - (section_nr_to_pfn(pnum)));
    BUILD_BUG_ON(SECTION_MAP_LAST_BIT > PFN_SECTION_SHIFT);
    BUG_ON(coded_mem_map & ~SECTION_MAP_MASK);
```

```
    return coded_mem_map;
}

static inline unsigned long section_nr_to_pfn(unsigned long sec)
{
    return sec << PFN_SECTION_SHIFT;
}
```

**section_nr_to_pfn(pnum)**：将内存段编号 pnum 转换为该段的第一个物理页的页帧号（PFN）。是该段所有物理页的 PFN 起始基准（段内后续页的 PFN 为 “起始 PFN + 页偏移”）。假设由 120 个物理内存，PFN 编号(内存系统中全局唯一)：0~119。分为 3 个 section：section0、section1、section2。

内存段编号 pnum 为 1 的段内第一个物理页的页帧号 PFN 就是 40 。

**mem_map**：当前内存段对应的 struct page 数组起始指针（内核虚拟地址）。

**mem_map - section_nr_to_pfn(pnum)**：计算 mem_map 指针与“内存段起始 PFN 的数值”之间的 偏移量。相当于把 “mem_map - section_start_pfn” 的固定偏移提前计算好，后续从 coded_mem_map 中提 struct page 的虚拟内存地址是只需一次加法就能得到 ，无需再计算段内偏移。

这与从 coded_mem_map 中解码对应内存段号的起始页的虚拟地址呼应起来了：

```
/*
 * Decode mem_map from the coded memmap
 */
struct page *sparse_decode_mem_map(unsigned long coded_mem_map, unsigned
long pnum)
{
    /* mask off the extra low bits of information */
    coded_mem_map &= SECTION_MAP_MASK;
    return ((struct page *)coded_mem_map) + section_nr_to_pfn(pnum);
}
```

在 SPARSEMEM 稀疏内存模型下 page_to_pfn 与 pfn_to_page 的计算逻辑发生了变化:

1、在 page_to_pfn 的转换中，首先需要通过 page_to_section 根据 struct page 结构定位到 mem_section 数组中具体的 section 结构。然后在通过 section_mem_map 定位到具体的 PFN。(***在 struct page 结构中有一个 unsigned long flags 属性，在 flag 的高位 bit 中存储着 page 所在 mem_section 数组中的索引，从而可以定位到所属 section。***)

2、在 pfn_to_page 的转换中，首先需要通过 __pfn_to_section 根据 PFN 定位到 mem_section 数组中具体的 section 结构。然后再通过 PFN 在 section_mem_map 数组中定位到具体的物理页 Page。(***PFN 的高位 bit 存储的是全局数组 mem_section 中的 section 索引，PFN 的低位 bit 存储的是 section_mem_map 数组中具体物理页***

*page 的索引。*)

PFN 的高位 bit 存储的是全局数组 mem_section 中的 section 索引，PFN 的低位 bit 存储的是 section_mem_map 数组中具体物理页 page 的索引。

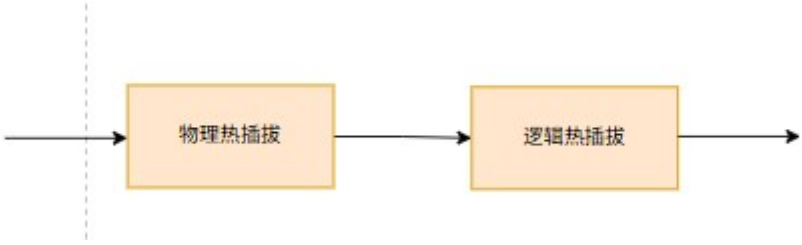
```
#elif defined(CONFIG_SPARSEMEM)
/*
 * Note: section's mem_map is encoded to reflect its start_pfn.
 * section[i].section_mem_map == mem_map's address - start_pfn;
 */
#define __page_to_pfn(pg)                    \
({  const struct page *__pg = (pg);            \
    int __sec = page_to_section(__pg);         \
    (unsigned long)(__pg -
__section_mem_map_addr(__nr_to_section(__sec))); \
})

#define __pfn_to_page(pfn)              \
({  unsigned long __pfn = (pfn);            \
    struct mem_section *__sec = __pfn_to_section(__pfn);    \
    __section_mem_map_addr(__sec) + __pfn;      \
})
#endif
```

可以看出 SPARSEMEM 稀疏内存模型已经完全覆盖了前两个内存模型的所有功能，因此稀疏内存模型可被用于所有内存布局的情况。

## 4.2 物理内存热插拔

在大规模的集群中，尤其是现在我们处于云原生的时代，为了实现集群资源的动态均衡，可以通过物理内存热插拔的功能实现集群机器物理内存容量的动态增减。集群的规模一大，那么物理内存出故障的几率也会大大增加，物理内存的热插拔对提供集群高可用性也是至关重要的。

从总体上来讲，内存的热插拔分为两个阶段：

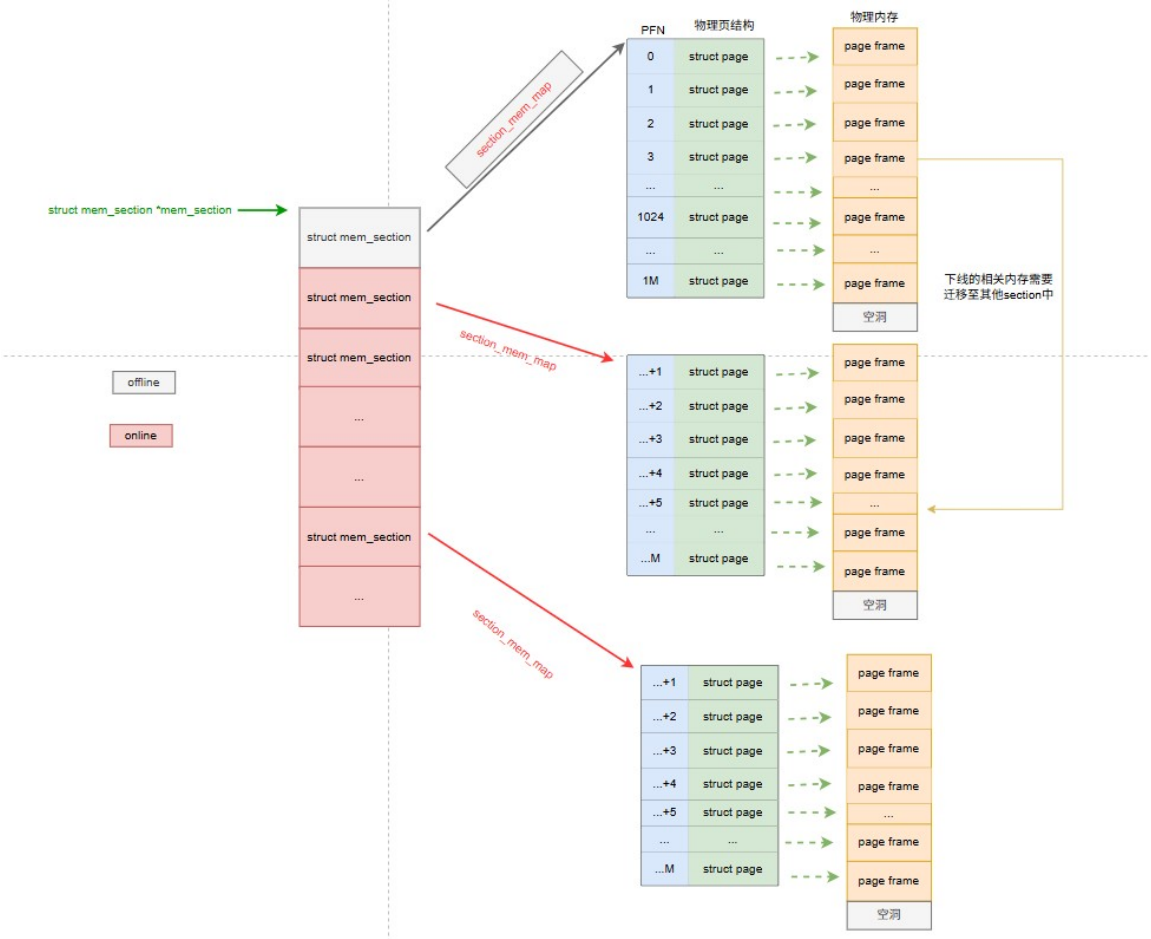



1、物理热插拔阶段：这个阶段主要是从物理上将内存硬件插入（hot-add），拔出（hot-remove）主板的过程，其中涉及到硬件和内核的支持。

2、逻辑热插拔阶段：这一阶段主要是由内核中的内存管理子系统来负责，涉及到的主要工作为：如何动态的上线启用（online）刚刚 hot-add 的内存，如何动态下线（offline）刚刚 hot-remove 的内存。

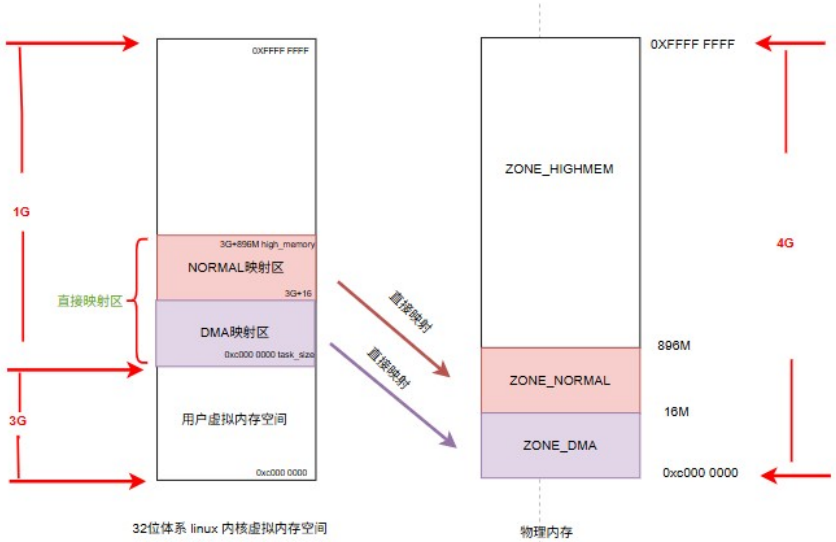
物理内存插入比较简单，困难的是物理内存的动态拔出，此时即将要被拔出的物理内存中可能已经为进程分配了物理页，如何妥善安置这些已经被分配的物理页是一个棘手的问题。

在 SPARSEMEM 内存模型**每个 mem_section** 都可以在系统运行时改变 offline ，online 状态，以便支持内存的热插拔（hotplug）功能。 当 mem_section offline 时，内核会把这部分内存隔离开，使得该部分内存不可再被使用，然后再把 mem_section 中已经分配的内存页迁移到其他 mem_section 的内存上. 。




这里会有一个问题，就是并非所有的物理页都可以迁移，因为迁移意味着物理内存地址的变化，而内存的热插拔应该对进程来说是透明的，所以这些迁移后的物理页映射的虚拟内存地址是不能变化的。

这一点在进程的用户空间是没有问题的，因为进程在用户空间访问内存都是根据虚拟内存地址通过页表找到对应的物理内存地址，这些迁移之后的物理页，虽然物理内存地址发生变化，但是内核通过修改相应页表中虚拟内存地址与物理内存地址之间的映射关系，可以保证虚拟内存地址不会改变。




但是在内核态的虚拟地址空间中，有一段直接映射区，在这段虚拟内存区域中虚拟地址与物理地址是直接映射的关系，虚拟内存地址直接减去一个固定的偏移量（0xC000 0000 ）就得到了物理内存地址。

直接映射区中的物理页的虚拟地址会随着物理内存地址变动而变动，因此这部分物理页是无法轻易迁移的，然而不可迁移的页会导致内存无法被拔除，因为无法妥善安置被拔出内存中已经为进程分配的物理页。那么内核是如何解决这个头疼的问题呢？

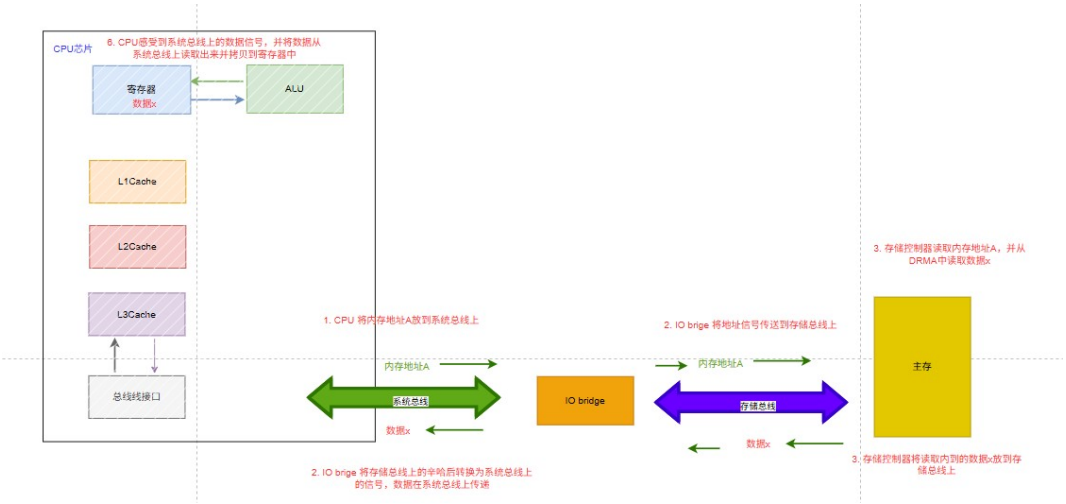
为了解决这一问题，将内存按照物理页是否可迁移，划分为：**不可迁移页**，**可回收页**，**可迁移页**。

然后在这些可能会被拔出的内存中只分配那些可迁移的内存页，这些信息会在内存初始化的时候被设置，这样一来那些不可迁移的页就不会包含在可能会拔出的内存中，当我们需要将这块内存热拔出时，因为里边的内存页全部是可迁移的，从而使内存可以被拔除。

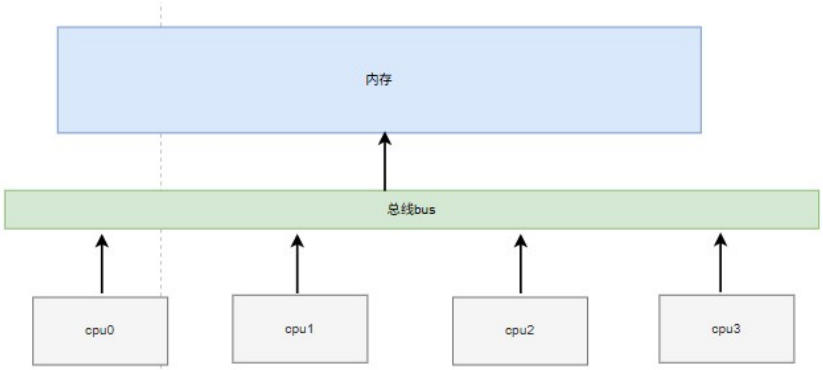
## 4.3 从 CPU 角度看物理内存架构

### 4.3.1 一致性内存访问 UMA 架构

单核 CPU 访问内存的架构图：




在 UMA 架构下，多核服务器中的多个 CPU 位于总线的一侧，所有的内存条组成一大片内存位于总线的另一侧，所有的 CPU 访问内存都要过总线，而且**距离都是一样**的，由于所有 CPU 对内存的访问距离都是一样的，所以在 UMA 架构下**所有 CPU 访问内存的速度都是一样的**。这种访问模式称为 **SMP（Symmetric multiprocessing）**，即**对称多处理器**。*（一致性是指同一个 CPU 对所有内存的访问的速度是一样的。即一致性内存访问 UMA（Uniform Memory Access））。*




是随着多核技术的发展，服务器上的 CPU 个数会越来越多，而 UMA 架构下所有 CPU 都是需要通过总线来访问内存的，这样总线很快就会成为性能瓶颈，主要体现在以下两个方面：

1、总线的带宽压力会越来越大，随着 CPU 个数的增多导致每个 CPU 可用带宽会减少

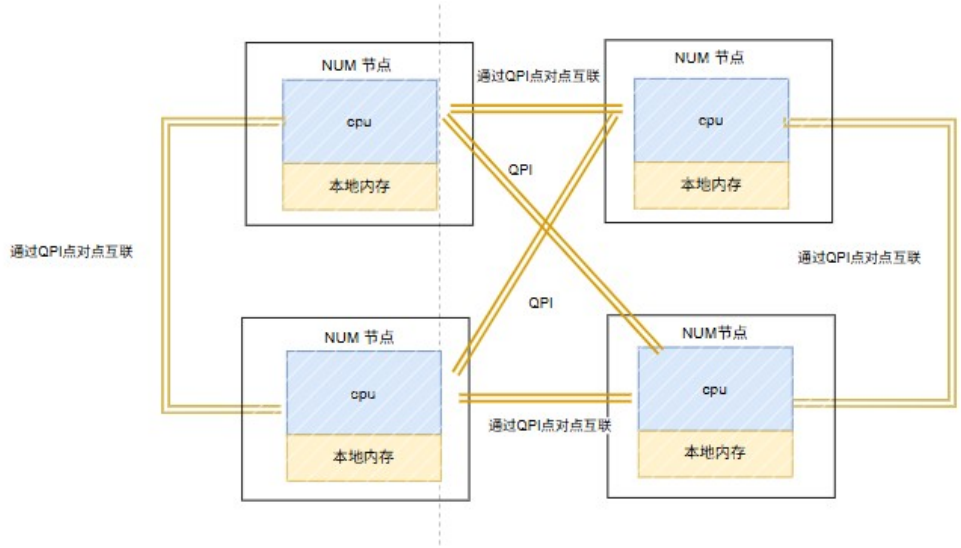
2、总线的长度也会因此而增加，进而增加访问延迟

UMA 架构的优点很明显就是结构简单，所有的 CPU 访问内存速度都是一致的，都必须经过总线。然而它的缺点，就是随着处理器核数的增多，总线的带宽压力会越来越大。解决办法就只能扩宽总线，然而成本十分高昂，未来可能仍然面临带宽压力。

为了解决以上问题，提高 CPU 访问内存的性能和扩展性，于是引入了一种新的架构：**非一致性内存访问 NUMA（Non-uniform memory access）**。

## 4.3.2 非一致性内存访问 NUMA 架构

在 NUMA 架构下，内存不是一整片，而是被划分成了一个一个的内存节点（NUMA 节点），每个 CPU 都有属于自己的本地内存节点，CPU 访问自己的本地内存不需要经过总线，因此访问速度是最快的。当 CPU 自己的本地内存不足时，CPU 就需要跨节点去访问其他内存节点，这种情况下 CPU 访问内存就会慢很多。




如上图所示，CPU 和它的本地内存组成了 NUMA 节点，CPU 与 CPU 之间通过 QPI（Intel QuickPath Interconnect）点对点完成互联，在 CPU 的本地内存不足的情况下，CPU 需要通过 QPI 访问远程 NUMA 节点上的内存控制器从而在远程内存节点上分配内存，这就导致了远程访问比本地访问多了额外的延迟开销（需要通过 QPI 遍历远程 NUMA 节点）。

**在 NUMA 架构下，只有 DISCONTIGMEM 非连续内存模型和 SPARSEMEM 稀疏内存模型是可用的。而 UMA 架构下，前面介绍的三种内存模型都可以配置使用。**

## 4.3.3 NUMA 的内存分配策略

NUMA 的内存分配策略是指在 NUMA 架构下 CPU 如何请求内存分配的相关策略，比如：是优先请求本地内存节点分配内存呢？还是优先请求指定的 NUMA 节点分配内存？是只能在本地内存节点分配呢？还是允许当本地内存不足的情况下可以请求远程 NUMA 节点分配内存？内存分配策略用下面的宏设置。

```
#define MPOL_DEFAULT     0
#define MPOL_PREFERRED   1
#define MPOL_BIND    2
#define MPOL_INTERLEAVE  3
```

**MPOL_DEFAUL：**优先在本地节点分配，当本地节点内存不足时，可以在远程节点分配内存
**MPOL_PREFERRED：**优先在指定节点分配内存，当指定节点内存不足时，选择离指定节点最近的节点分配内存
**MPOL_BIND：**必须在绑定的节点进行内存分配，如果内存不足，则进行 swap

**MPOL_INTERLEAVE：** 本地节点和远程节点均可允许分配内存

应用程序中通过 libnuma 共享库中的 API 调用 set_mempolicy 接口设置进程的内存分配策略。

```
#include <numaif.h>

long set_mempolicy(int mode, const unsigned long *nodemask,
                   unsigned long maxnode);
```

mode：指定 NUMA 内存分配策略。

nodemask：指定 NUMA 节点 Id。

maxnode：指定最大 NUMA 节点 Id，用于遍历远程节点，实现跨 NUMA 节点分配内存。

**libnuma 共享库 API 文档：**
https://man7.org/linux/man-pages/man3/numa.3.html#top_of_pageset_mempolicy

**接口文档：**https://man7.org/linux/man-pages/man2/set_mempolicy.2.html

## 4.3.4 内核如何管理 NUMA 节点

无论是 NUMA 架构还是 UMA 架构在内核中都是使用相同的数据结构来组织管理的，在内核的内存管理模块中会把 UMA 架构当做只有一个 NUMA 节点的伪 NUMA 架构。这样一来这两种架构模式就在内核中被统一管理起来。**NUMA 节点中可能会包含多个 CPU，这些 CPU 均是物理 CPU.**

### 4.3.4.1 内核如何统一组织 NUMA 节点

内核中使用了 **struct pglist_data** 这样的一个数据结构来描述 NUMA 节点。

```
typedef struct pglist_data {

        ...... 省略......
        //node 中包含的 zone 的集合数据
        struct zone node_zones[MAX_NR_ZONES];
        //对所有 node 的 zone 的引用
        struct zonelist node_zonelists[MAX_ZONELISTS];

        #ifdef CONFIG_FLATMEM   /* means !SPARSEMEM */
            //指向 NUMA 节点内管理所有物理页 page 的数组
            struct page *node_mem_map;
```

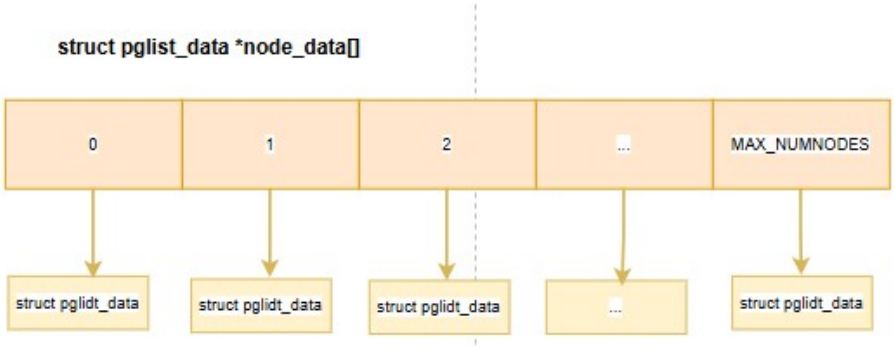
```
        #ifdef CONFIG_PAGE_EXTENSION
            struct page_ext *node_page_ext;
        #endif
    //NUMA 节点内第一个物理页的 pfn
    unsigned long node_start_pfn;
    //// NUMA 节点内所有可用的物理页个数（不包含内存空洞）
    unsigned long node_present_pages;
    unsigned long node_spanned_pages;//node 包含的物理内存区间，包含中间的
hole

    int node_id;//NUMA 节点 id
    ...... 省略......
}
```

**node_id** 表示 NUMA 节点的 id。
struct page 类型的数组 **node_mem_map** 中包含了 NUMA 节点内的所有的物理内存页。

**node_start_pfn** 指向 NUMA 节点内第一个物理页的 PFN，系统中所有 NUMA 节点中的物理页都是依次编号的，每个物理页的 PFN 都是全局唯一的（不只是其所在 NUMA 节点内唯一）。
**node_present_pages** 用于统计 NUMA 节点内所有真正可用的物理页面数量（不包含内存空洞）。由于 NUMA 节点内包含的物理内存并不总是连续的，可能会包含一些内存空洞。

**node_spanned_pages** 则是用于统计 NUMA 节点内所有的内存页，包含不连续的物理内存地址（内存空洞）的页面数。

内核使用了一个大小为 MAX_NUMNODES ，类型为 struct pglist_data 的全局数组 node_data[] 来管理所有的 NUMA 节点。全局数组 node_data[] 定义在文件 /arch/arm64/include/asm/mmzone.h 中，



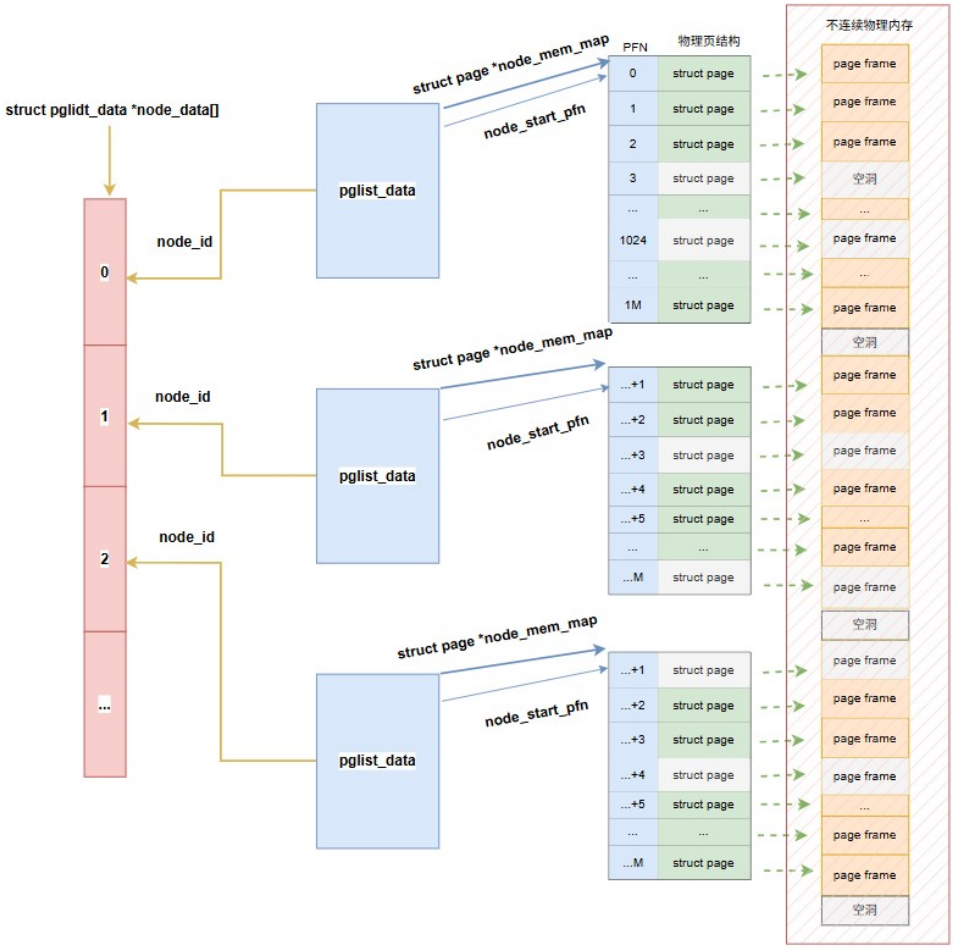

```
#ifdef CONFIG_NUMA

#include <asm/numa.h>
extern struct pglist_data *node_data[];
```

```
#define NODE_DATA(nid)      (node_data[(nid)])
#endif
```

NODE_DATA(nid) 宏可以通过 NUMA 节点的 nodeId，找到对应的 struct pglist_data 结构。node_data[] 数组大小 MAX_NUMNODES 定义在 /include/linux/numa.h 文件中：

```
#ifdef CONFIG_NODES_SHIFT
#define NODES_SHIFT     CONFIG_NODES_SHIFT
#else
#define NODES_SHIFT     0
#endif
#define MAX_NUMNODES    (1 << NODES_SHIFT)
```

UMA 架构下 NODES_SHIFT 为 0 ，所以内核中只用一个 NUMA 节点来管理所有物理内存。

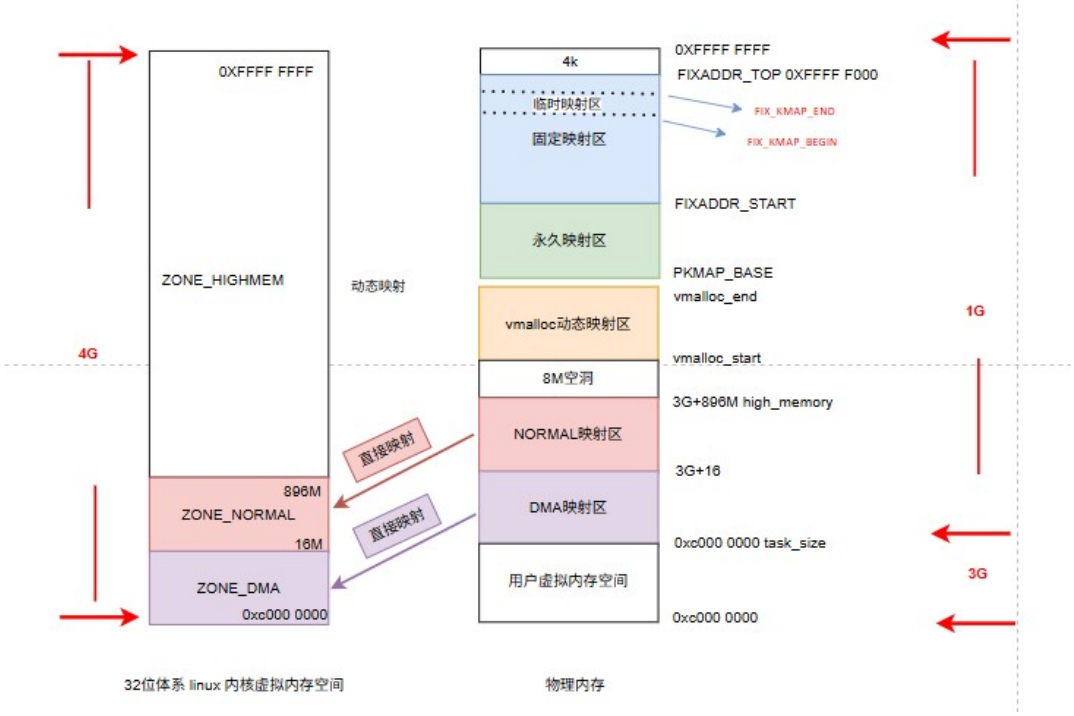


### 4.3.4.2 NUMA 节点内的物理内存区域划分

内核对物理内存的管理都是以页为最小单位来管理的，每页默认 4K 大小，理想状况下任何种类的数据都可以存放在任何页框中，没有什么限制。比如：存放内核数据，用户数据，磁盘缓冲数据等。

但是实际的计算机体系结构受到硬件方面的制约，间接导致限制了页框的使用方式。比如在 X86 体系结构下，ISA 总线的 DMA（直接内存存取）控制器，只能对内存的前 16M 进行寻址，这就导致了 ISA 设备不能在整个 32 位地址空间中执行 DMA，只能使用物理内存的前 16M 进行 DMA 操作。

因此直接映射区的前 16M 专门让内核用来为 DMA 分配内存，这块 16M 大小的内存区域我们称之为 ZONE_DMA。用于 DMA 的内存必须从 ZONE_DMA 区域中分配。




直接映射区中剩下的部分也就是从 16M 到 896M（不包含 896M）这段区域，我们称之为 **ZONE_NORMAL**。从字面意义上我们可以了解到，这块区域包含的就是正常的页框（没有任何使用限制）。

ZONE_NORMAL 由于也是属于直接映射区的一部分，对应的物理内存 16M 到 896M 这段区域也是被直接映射至内核态虚拟内存空间中的 3G + 16M 到 3G + 896M 这段虚拟内存上。

物理内存 896M 以上的区域被内核划分为 ZONE_HIGHMEM 区域，我们称之为**高端内存**。

由于内核虚拟内存空间中的前 896M 虚拟内存已经被直接映射区所占用，而在 32 体系结构下内核虚拟内存空间总共也就 1G 的大小，这样一来内核剩余可用的虚拟内存空间就变为了 1G - 896M = 128M。

显然物理内存中剩下的这 3200M 大小的 ZONE_HIGHMEM 区域无法继续通过直接映射的方式映射到这 128M 大小的虚拟内存空间中。**物理内存中的 ZONE_HIGHMEM 区域就只能采用动态映射的方式映射到 128M 大小的内核虚拟内存空间中**，也就是说只能动态的

一部分一部分的分批映射，先映射正在使用的这部分，使用完毕解除映射，接着映射其他部分。

内核会根据各个物理内存区域的功能不同，将 NUMA 节点内的物理内存主要划分为以下四个物理内存区域：

1、ZONE_DMA：用于那些无法对全部物理内存进行寻址的硬件设备，进行 DMA 时的内存分配。例如前边介绍的 ISA 设备只能对物理内存的前 16M 进行寻址。该区域的长度依赖于具体的处理器类型。

2、ZONE_DMA32：与 ZONE_DMA 区域类似，该区域内的物理页面可用于执行 DMA 操作，不同之处在于该区域是提供给 32 位设备（只能寻址 4G 物理内存）执行 DMA 操作时使用的。该区域只在 64 位系统中起作用，因为只有在 64 位系统中才会专门为 32 位设备提供专门的 DMA 区域。

3、ZONE_NORMAL：这个区域的物理页都可以直接映射到内核中的虚拟内存，由于是线性映射，**内核可以直接进行访问**。

4、ZONE_HIGHMEM：这个区域包含的物理页就是我们说的高端内存，内核不能直接访问这些物理页，这些物理页需要动态映射进内核虚拟内存空间中（非线性映射）。该区域只在 32 位系统中才会存在，因为 64 位系统中的内核虚拟内存空间太大了（128T），都可以进行直接映射。

以上这些物理内存区域的划分定义在 /include/linux/mmzone.h 中，

```
enum zone_type {

#ifdef CONFIG_ZONE_DMA
    ZONE_DMA,
#endif
#ifdef CONFIG_ZONE_DMA32
    ZONE_DMA32,
#endif
    ZONE_NORMAL,
#ifdef CONFIG_HIGHMEM
    ZONE_HIGHMEM,
#endif
    ZONE_MOVABLE,
#ifdef CONFIG_ZONE_DEVICE
    ZONE_DEVICE,
#endif
     // 充当结束标记，在内核中想要迭代系统中所有内存域时，会用到该常量
    __MAX_NR_ZONES

};
```

**ZONE_DEVICE** 是为支持热插拔设备而分配的非易失性内存（ Non Volatile Memory ），也可用于内核崩溃时保存相关的调试信息。

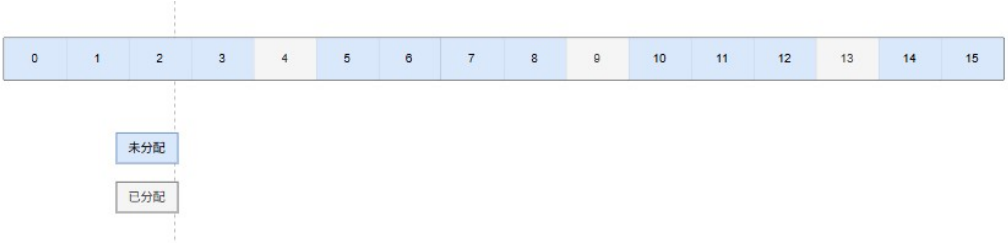
**ZONE_MOVABLE** 是内核定义的一个**虚拟内存区域**，该区域中的物理页**可以来自于上边**

**介绍的几种真实的物理区域**。该区域中的页全部都是可以迁移的，主要是为了防止内存碎片和支持内存的热插拔。

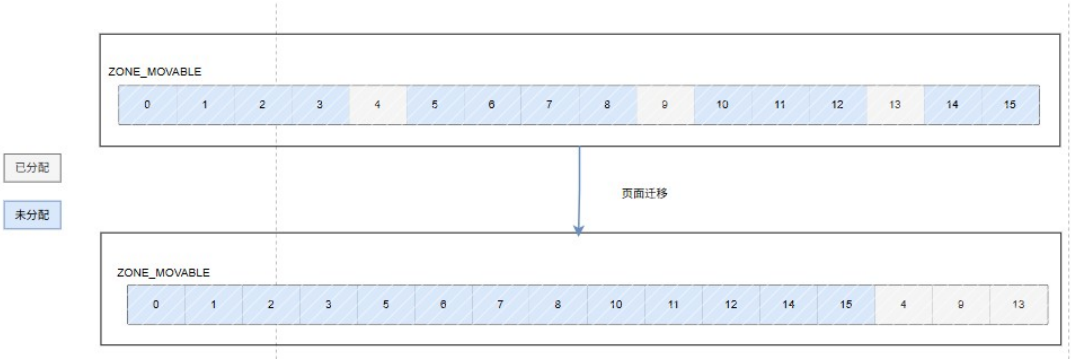
**划分 ZONE_MOVABLE 区域的原因：**

随着系统的运行会伴随着不同大小的物理内存页的分配和释放，这种内存不规则的分配释放随着系统的长时间运行就会导致内存碎片，内存碎片会使得系统在明明有足够内存的情况下，依然无法为进程分配合适的内存。

假如现在系统一共有 16 个物理内存页，当前系统只是分配了 3 个物理页，那么在当前系统中还剩余 13 个物理内存页的情况下，如果内核想要分配 8 个连续的物理页的话，就会由于内存碎片的存在导致分配失败。（只能分配最多 4 个连续的物理页）。




如果这些物理页处于 ZONE_MOVABLE 区域，它们就可以被迁移，内核可以通过迁移页面来避免内存碎片的问题：




内核通过迁移页面来规整内存，这样就可以避免内存碎片，从而得到一大片连续的物理内存，以满足内核对大块连续内存分配的请求。**这就是内核需要根据物理页面是否能够迁移的特性，而划分出 ZONE_MOVABLE 区域的目的**。

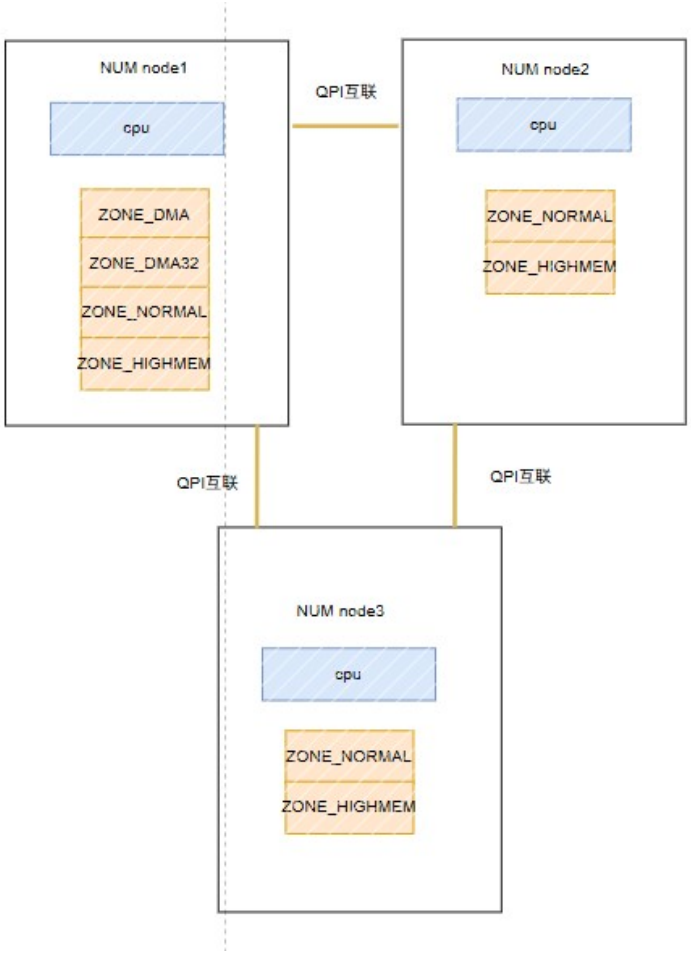
struct pglist_data 结构中内核如何在 NUMA 节点中组织这些划分出来的内存区域？

```
typedef struct pglist_data {

        ...... 省略......
        //node 中包含的 zone 的集合数据
        struct zone node_zones[MAX_NR_ZONES];
        //对所有 node 的 zone 的引用
        struct zonelist node_zonelists[MAX_ZONELISTS];
        // NUMA 节点中的物理内存区域个数
        int nr_zones;
}
```

**nr_zones** 用于统计 NUMA 节点内包含的物理内存区域个数，**不是每个 NUMA 节点都会包含以上介绍的所有物理内存区域，NUMA 节点之间所包含的物理内存区域个数是不一样的。**

只有第一个 NUMA 节点可以包含所有的物理内存区域，其它的节点并不能包含所有的区域类型，因为有些内存区域比如：ZONE_DMA，ZONE_DMA32 必须从物理内存的起点开始。这些在物理内存开始的区域可能已经被划分到第一个 NUMA 节点了，后面的物理内存才会被依次划分给接下来的 NUMA 节点。因此后面的 NUMA 节点并不会包含 ZONE_DMA，ZONE_DMA32 区域。

ZONE_NORMAL、ZONE_HIGHMEM 和 ZONE_MOVABLE 是可以出现在所有 NUMA 节点上的。




**node_zones[MAX_NR_ZONES]** 数组包含了 NUMA 节点中的所有物理内存区域，物理内存区域在内核中的数据结构是 struct zone 。

**node_zonelists[MAX_ZONELISTS]** 是 struct zonelist 类型的数组，它包含了备用 NUMA 节点和这些备用节点中的物理内存区域。备用节点是按照访问距离的远近，依次排列在 node_zonelists 数组中，数组第一个备用节点是访问距离最近的，这样当本节点内存不足时，可以从备用 NUMA 节点中分配内存。

#### 4.3.4.3 NUMA 节点中的内存规整与回收

内核会为**每个 NUMA 节点**分配一个 kswapd 进程用于回收不经常使用的页面，还会为每个 NUMA 节点分配一个 kcompactd 进程用于内存的规整避免内存碎片。

```
typedef struct pglist_data {
              ......
    wait_queue_head_t kswapd_wait;
             ......
    struct task_struct *kswapd;
               ......
#ifdef CONFIG_COMPACTION
               ......
    wait_queue_head_t kcompactd_wait;
    struct task_struct *kcompactd;
#endif
               ......

}
```

**struct task_struct *kswapd** 属性用于指向内核为 NUMA 节点分配的 kswapd 进程。
**kswapd_wait** 用于 kswapd 进程周期性回收页面时使用到的等待队列。
**struct task_struct *kcompactd** 用于指向内核为 NUMA 节点分配的 kcompactd 进程。
**kcompactd_wait** 用于 kcompactd 进程周期性规整内存时使用到的等待队列。

#### 4.3.4.4 NUMA 节点的状态 node_states

如果系统中的 NUMA 节点多于一个，内核会维护一个位图 node_states，用于维护各个 NUMA 节点的状态信息。如果系统中只有一个 NUMA 节点，则没有节点位图。

节点位图以及节点的状态掩码值定义在 /include/linux/nodemask.h 文件中，

```
typedef struct { DECLARE_BITMAP(bits, MAX_NUMNODES); } nodemask_t;
extern nodemask_t node_states[NR_NODE_STATES];
```

节点的状态可通过以下掩码表示：

```
/*
 * Bitmasks that are kept for all the nodes.
 */
enum node_states {
    N_POSSIBLE,     /* The node could become online at some point */
    N_ONLINE,       /* The node is online */
    N_NORMAL_MEMORY,    /* The node has regular memory */
```

```
#ifdef CONFIG_HIGHMEM
    N_HIGH_MEMORY,      /* The node has regular or high memory */
#else
    N_HIGH_MEMORY = N_NORMAL_MEMORY,
#endif
    N_MEMORY,       /* The node has memory(regular, high, movable) */
    N_CPU,      /* The node has one or more cpus */
    N_GENERIC_INITIATOR,    /* The node has one or more Generic Initiators
*/
    NR_NODE_STATES
};
```

**N_POSSIBLE** 表示 NUMA 节点在某个时刻可以变为 online 状态，**N_ONLINE** 表示 NUMA 节点当前的状态为 online 状态。在稀疏内存模型中，NUMA 节点的状态可以在系统运行的过程中随时切换 online ，offline 的状态，用来支持内存的热插拔。

**N_NORMAL_MEMORY** 表示节点没有高端内存，只有 ZONE_NORMAL 内存区域。

**N_HIGH_MEMORY** 表示节点有 ZONE_NORMAL 内存区域或者有 ZONE_HIGHMEM 内存区域。

**N_MEMORY** 表示节点有 ZONE_NORMAL，ZONE_HIGHMEM，ZONE_MOVABLE 内存区域。

**N_CPU** 表示节点包含一个或多个 CPU。

内核还提供了两个辅助函数用于设置或者清除指定节点的特定状态：

```
static inline void node_set_state(int node, enum node_states state)
static inline void node_clear_state(int node, enum node_states state)
```

内核提供了 for_each_node_state 宏用于迭代处于特定状态的所有 NUMA 节点。

```
#define for_each_node_state(__node, __state) \
    for_each_node_mask((__node), node_states[__state])
```

for_each_online_node 用于迭代所有 online 的 NUMA 节点：

```
#define for_each_online_node(node) for_each_node_state(node, N_ONLINE)
```

#### 4.3.4.5 内核管理 NUMA 节点中的物理内存区域

由于实际的计算机体系结构受到硬件方面的制约，间接限制了页框的使用方式。于是内核会根据各个物理内存区域的功能不同，将 NUMA 节点内的物理内存划分为：

ZONE_DMA，

ZONE_DMA32，

ZONE_NORMAL，

ZONE_HIGHMEM

这几个物理内存区域。ZONE_MOVABLE 区域是内核从逻辑上的划分，区域中的物理页面来自于上述几个内存区域，目的是避免内存碎片和支持内存热插拔.

内核中用于描述和管理 NUMA 节点中的物理内存区域的结构体是 struct zone。 struct zone 结构体的整体布局情况：

```
struct zone {

    ...省略...
    CACHELINE_PADDING(_pad1_);

    ...省略...
    CACHELINE_PADDING(_pad2_);
    ...省略...
    CACHELINE_PADDING(_pad3_);
    ...省略...
}____cacheline_internodealigned_in_smp;
```

struct zone 结构体在内核中是一个访问非常频繁的结构体，在多处理器系统中，会有不同的 CPU 同时大量频繁的访问 struct zone 结构体中的不同字段。因此内核对 struct zone 结构体的设计是相当考究的，将这些频繁访问的字段信息归类为 4 个部分，并通过 ZONE_PADDING 来分割。

目的是通过 ZONE_PADDING 来填充字节，将这四个部分，分别填充到不同的 CPU 高速缓存行（cache line）中，使得它们各自独占 cache line，提高访问性能。内核会把 NUMA 节点中的物理内存区域顶多划分为 ZONE_DMA，ZONE_DMA32，ZONE_NORMAL，ZONE_HIGHMEM 这几个物理内存区域。因此 struct zone 的实例在内核中会相对比较少，通过 ZONE_PADDING 填充字节，带来的 struct zone 结构体实例内存占用增加是可以忽略不计的。

**____cacheline_internodealigned_in_smp** 编译器关键字实现最优的高速缓存行对齐方式。

struct zone 结构体中一些字段：

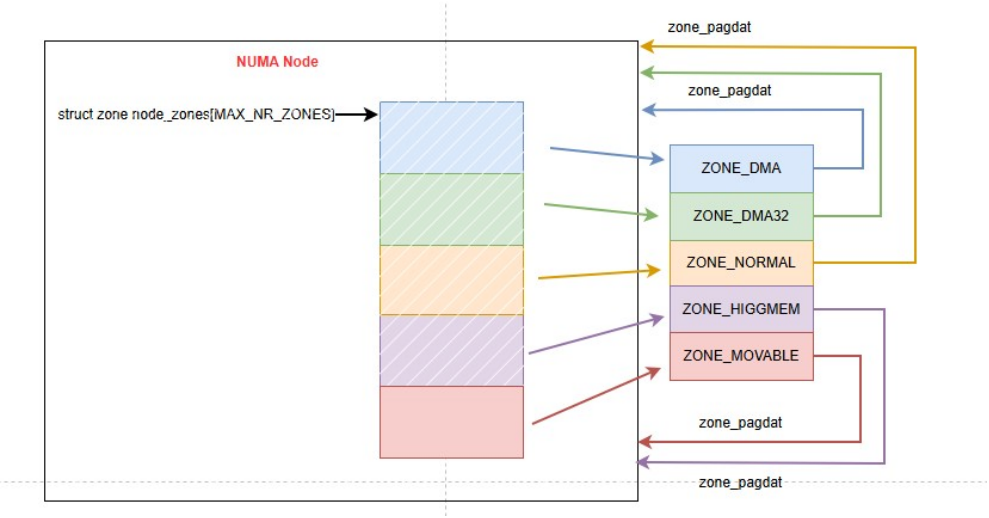
```
struct zone {
   // 防止并发访问该内存区域
    spinlock_t      lock;
    // 内存区域名称：Normal ，DMA，HighMem
    const char      *name;
    // 指向该内存区域所属的 NUMA 节点
    struct pglist_data  *zone_pgdat;
    // 属于该内存区域中的第一个物理页 PFN
    unsigned long       zone_start_pfn;
    // 伙伴系统所管理的物理内存页个数
    atomic_long_t       managed_pages;
    // 该内存区域中所有的物理页个数（包含内存空洞）
    unsigned long       spanned_pages;
     // 该内存区域所有可用的物理页个数（不包含内存空洞）
    unsigned long       present_pages;
    // 伙伴系统的核心数据结构
    struct free_area    free_area[NR_PAGE_ORDERS];
```

```
    // 该内存区域内存使用的统计信息
    atomic_long_t       vm_stat[NR_VM_ZONE_STAT_ITEMS];
}
```

struct zone 是会被内核频繁访问的一个结构体，在多核处理器中，多个 CPU 会并发访问 struct zone，为了防止并发访问，内核使用了一把 **spinlock_t lock 自旋锁**来防止并发错误以及不一致。

**name** 属性会根据该内存区域的类型不同保存内存区域的名称，比如：Normal ，DMA，HighMem 等。

介绍 NUMA 节点的描述符 struct pglist_data 的时候提到，pglist_data 通过 元素 struct zone node_zones[MAX_NR_ZONES] 将 NUMA 节点中划分的物理内存区域连接起来。这些物理内存区域也会通过 struct zone 中的 zone_pgdat 指向自己所属的 NUMA 节点。



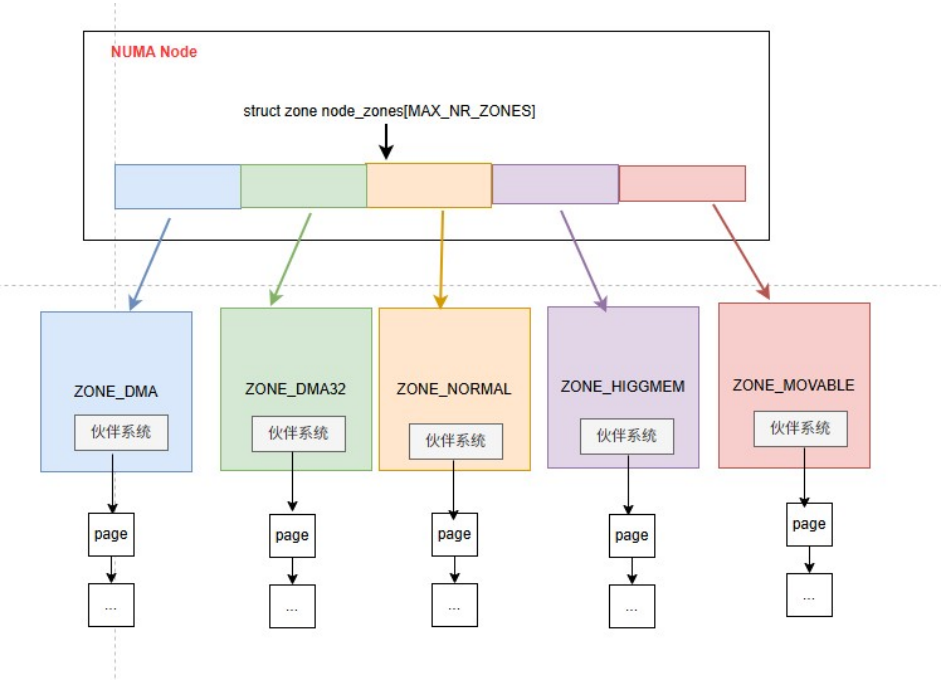

NUMA 节点 struct pglist_data 结构中的 **node_start_pfn** 指向 NUMA 节点内第一个物理页的 PFN。同理物理内存区域 struct zone 结构中的 **zone_start_pfn** 指向的是该内存区域内所管理的第一个物理页面 PFN 。

**spanned_pages** 表示该内存区域内所有的物理页总数（包含内存空洞），通过 spanned_pages = zone_end_pfn - zone_start_pfn 计算得到。

**present_pages** 则表示该内存区域内所有实际可用的物理页面总数（不包含内存空洞），通过 present_pages = spanned_pages - absent_pages(pages in holes) 计算得到

在 NUMA 架构下，物理内存被划分成了一个一个的内存节点（NUMA 节点），在每个 NUMA 节点内部又将其所管理的物理内存按照功能不同划分成了不同的内存区域，每个内存区域管理一片用于具体功能的物理内存，而内核会为每一个内存区域分配一个伙伴系统用于管理该内存区域下物理内存的分配和释放。

物理内存在内核中管理的层级关系为：Node -> Zone -> page




struct zone 结构中的 **managed_pages** 用于表示该内存区域内被伙伴系统所管理的物理页数量。

数组 **free_area[MAX_ORDER]** 是伙伴系统的核心数据结构。

**vm_stat** 维护了该内存区域物理内存的使用统计信息

## 4.4 物理内存区域中的预留内存

每个物理内存区域 struct zone 为操作系统预留了一部分内存，这部分预留的物理内存用于内核的一些核心操作，这些操作无论如何是不允许内存分配失败的。

内核中关于内存分配的场景无外乎有两种方式：

1、当进程请求内核分配内存时，如果此时内存比较充裕，那么进程的请求会被立刻满足，如果此时内存已经比较紧张，内核就需要将一部分不经常使用的内存进行回收，从而腾出一部分内存满足进程的内存分配的请求，在这个回收内存的过程中，进程会一直**阻塞等待**。

2、另一种内存分配场景，进程是**不允许阻塞**的，内存分配的请求必须马上得到满足，比如执行中断处理程序或者执行持有自旋锁等临界区内的代码时，进程就不允许睡眠，因为中断程序无法被重新调度。这时就需要内核提前为这些核心操作预留一部分内存，当内存紧张时，可以使用这部分预留的内存给这些操作分配。

```
struct zone {

        ...省略...
        unsigned long nr_reserved_highatomic;
        long lowmem_reserve[MAX_NR_ZONES];
        ...省略...
}
```

**nr_reserved_highatomic** 表示的是该内存区域内预留内存的大小，范围为 128 到 65536 KB 之间。
**lowmem_reserve** 数组则是用于规定每个内存区域必须为自己保留的物理页数量，防止更高位的内存区域对自己的内存空间进行过多的侵占挤压。
**低位内存区域**和**高位内存区域**是按照物理内存地址从低到高进行排列布局。根据物理内存地址的高低，低位内存区域到高位内存区域的顺序依次是：ZONE_DMA，ZONE_DMA32，ZONE_NORMAL，ZONE_HIGHMEM。

**高位内存区域为什么会对低位内存区域进行挤压呢 ？**

一些用于特定功能的物理内存必须从特定的内存区域中进行分配，比如外设的 DMA 控制器就必须从 ZONE_DMA 或者 ZONE_DMA32 中分配内存。但是一些用于常规用途的物理内存则可以从多个物理内存区域中进行分配，当 ZONE_HIGHMEM 区域中的内存不足时，内核可以从 ZONE_NORMAL 进行内存分配，ZONE_NORMAL 区域内存不足时可以进一步降级到 ZONE_DMA 区域进行分配。

而低位内存区域中的内存总是宝贵的，内核肯定希望这些用于常规用途的物理内存从常规内存区域中进行分配，这样能够节省 ZONE_DMA 区域中的物理内存保证 DMA 操作的内存使用需求，但是如果内存很紧张了，高位内存区域中的物理内存不够用了，那么内核就会去占用挤压其他内存区域中的物理内存从而满足内存分配的需求。

内核不会允许高位内存区域对低位内存区域的无限制挤压占用，因为毕竟低位内存区域有它特定的用途，所以每个内存区域会给自己预留一定的内存，防止被高位内存区域挤压占用。而每个内存区域为自己预留的这部分内存就存储在 lowmem_reserve 数组中。**每个内存区域是按照一定的比例来计算自己的预留内存。**

**每个内存区域如何根据各自的 lowmem_reserve_ratio 来计算各自区域中的预留内存大小呢？**

以 ZONE_DMA，ZONE_NORMAL，ZONE_HIGHMEM 这三个物理内存区域举例，它们的 lowmem_reserve_ratio 分别为 256，32，0。它们的大小分别是：8M，64M，256M，按照每页大小 4K 计算它们区域里包含的物理页个数分别为：2048, 16384, 65536。
ZONE_DMA 为防止被 ZONE_NORMAL 挤压侵占，而为自己预留的物理内存页为：
16384 / 256 = 64。

ZONE_DMA 为防止被 ZONE_HIGHMEM 挤压侵占而为自己预留的物理内存页为：
(65536 + 16384) / 256 = 320。

ZONE_NORMAL 为防止被 ZONE_HIGHMEM 挤压侵占而为自己预留的物理内存页为：
65536 / 32 = 2048。

## 4.5 物理内存区域中的水位线

内存资源是系统中最宝贵的系统资源，是有限的。当内存资源紧张的时候，系统的应对方法无非就是三种：
1、产生 OOM，内核直接将系统中占用大量内存的进程，将 OOM 优先级最高的进程干掉，

释放出这个进程占用的内存供其他更需要的进程分配使用。
2、内存回收，将不经常使用到的内存回收，腾挪出来的内存供更需要的进程分配使用。
3、内存规整，将可迁移的物理页面进行迁移规整，消除内存碎片。从而获得更大的一片连续物理内存空间供进程分配。

内核将物理内存划分成一页一页的单位进行管理（每页 4K 大小）。内存回收的单位也是按页来的。在内核中，物理内存页有两种类型，针对这两种类型的物理内存页，内核会有不同的回收机制。

**第一种就是文件页**，所谓文件页就是其物理内存页中的数据来自于磁盘中的文件，当我们进行文件读取的时候，内核会根据局部性原理将读取的磁盘数据缓存在 page cache 中，page cache 里存放的就是文件页。当进程再次读取读文件页中的数据时，内核直接会从 page cache 中获取并拷贝给进程，省去了读取磁盘的开销。

对于文件页的回收通常会比较简单，因为文件页中的数据来自于磁盘，所以当回收文件页的时候直接回收就可以了，当进程再次读取文件页时，再从磁盘中重新读取。

**另外一种物理页类型是匿名页**，所谓匿名页就是它背后并没有一个磁盘中的文件作为数据来源，匿名页中的数据都是通过进程运行过程中产生的，比如我们应用程序中动态分配的堆内存。

当内存资源紧张需要对不经常使用的那些匿名页进行回收时，因为匿名页的背后没有一个磁盘中的文件做依托，所以匿名页不能像文件页那样直接回收，无论匿名页是不是脏页，都需要先将匿名页中的数据先保存在磁盘空间中，然后在对匿名页进行回收。

把释放出来的这部分内存分配给更需要的进程使用，当进程再次访问这块内存时，在重新把之前匿名页中的数据从磁盘空间中读取到内存就可以了，而这块磁盘空间可以是单独的一片磁盘分区（Swap 分区）或者是一个特殊的文件（Swap 文件）。**匿名页的回收机制就是我们经常看到的 Swap 机制**。

所谓的页面换出就是在 Swap 机制下，当内存资源紧张时，内核会把不经常使用的这些匿名页中的数据写入到 Swap 分区或者 Swap 文件中。从而释放这些数据所占用的内存空间。

综上所述，物理内存区域中的内存回收分为：**文件页回收（通过 pflush 内核线程）**和**匿名页回收（通过 kswapd 内核进程）**。Swap 机制主要针对的是匿名页回收。

Linux 提供了一个 swappiness 的内核选项，取值范围为 0 到 100，默认为 60。swappiness 用于表示 Swap 机制的积极程度，数值越大，Swap 的积极程度越高，内核越倾向于回收匿名页。数值越小，Swap 的积极程度越低。内核就越倾向于回收文件页。

swappiness 只是表示 Swap 积极的程度，当内存非常紧张的时候，即使将 swappiness 设置为 0 ，也还是会发生 Swap 的。

**内核中如何定义内存紧张？**

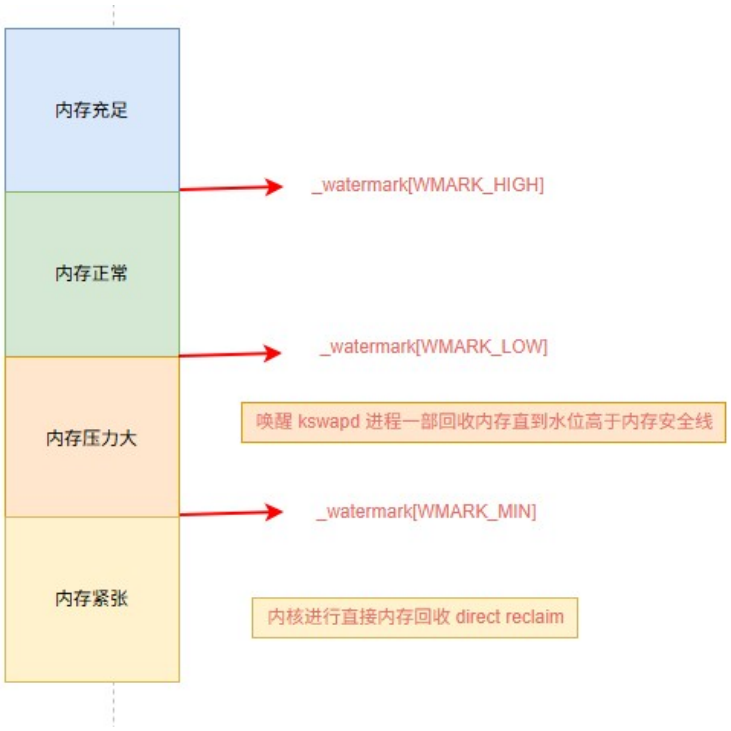
物理内存区域中的水位线。内核会为**每个 NUMA 节点中的每个物理内存区域**定制三条用于指示内存容量的水位线，分别是：
WMARK_MIN（页最小阈值），
WMARK_LOW （页低阈值），
WMARK_HIGH（页高阈值）。

这三条水位线定义在 /include/linux/mmzone.h 中：

```
enum zone_watermarks {
    WMARK_MIN,
    WMARK_LOW,
    WMARK_HIGH,
    WMARK_PROMO,
    NR_WMARK
};

#define min_wmark_pages(z) (z->_watermark[WMARK_MIN] + z->watermark_boost)
#define low_wmark_pages(z) (z->_watermark[WMARK_LOW] + z->watermark_boost)
#define high_wmark_pages(z) (z->_watermark[WMARK_HIGH] +
z->watermark_boost)
```

三条水位线对应的 watermark 数值存储在每个物理内存区域 struct zone 结构中的 **_watermark[NR_WMARK]** 数组中。

```
struct zone {
    /* zone watermarks, access with *_wmark_pages(zone) macros */
    unsigned long _watermark[NR_WMARK]; // 物理内存区域中的水位线
    // 优化内存碎片对内存分配的影响，可以动态改变内存区域的基准水位线。
    unsigned long watermark_boost;
    ...省略...
}____cacheline_internodealigned_in_smp;
```



当该物理内存区域的剩余内存容量高于 _watermark[WMARK_HIGH] 时，说明此时该物理内存区域中的内存容量非常充足，内存分配完全没有压力。

当剩余内存容量在 _watermark[WMARK_LOW] 与_watermark[WMARK_HIGH] 之间时，说明此时内存有一定的消耗但是还可以接受，能够继续满足进程的内存分配需求。

当剩余内容容量在 _watermark[WMARK_MIN] 与 _watermark[WMARK_LOW] 之间时，说明此时内存容量已经有点危险了，内存分配面临一定的压力，但是还可以满足进程的内存分配要求，当给进程分配完内存之后，就会**唤醒 kswapd 进程开始内存回收（此时请求进程本身不会被阻塞，由内核的 kswapd 进程异步回收内存）**，直到剩余内存高于 _watermark[WMARK_HIGH] 为止。

当剩余内容容量低于 _watermark[WMARK_MIN] 时，说明此时的内容容量已经非常危险了，如果进程在这时请求内存分配，**内核就会进行直接内存回收，这时请求进程会同步阻塞等待**，直到内存回收完毕。

位于 _watermark[WMARK_MIN] 以下的内存容量是预留给内核在紧急情况下使用的。

### 4.5.1 计算 min_free_kbytes

WMARK_MIN，WMARK_LOW ，WMARK_HIGH 这三个水位线的数值是通过内核参数 /proc/sys/vm/min_free_kbytes 为基准分别计算出来的。通常情况下 WMARK_LOW 的值是 WMARK_MIN 的 1.25 倍，WMARK_HIGH 的值是 WMARK_LOW 的 1.5 倍。而 WMARK_MIN 的数值就是由这个内核参数 min_free_kbytes 来决定的。(64 位系统中内存区域水位线的计算，在 64 位系统中没有高端内存 ZONE_HIGHMEM 区域。)

min_free_kbytes 表示系统**全局最小空闲页阈值（KB 单位）**，用户可通过 sysctl 动态调整，决定系统 “保底空闲内存” 规模。

min_free_kbytes 的计算逻辑定义在内核文件 /mm/page_alloc.c 的 init_per_zone_wmark_min 方法中。

```
int __meminit init_per_zone_wmark_min(void)
{
    //计算 min_free_kbytes
    calculate_min_free_kbytes();
    //计算内存区域内的三条水位线
    setup_per_zone_wmarks();

    refresh_zone_stat_thresholds();
    //计算内存区域的预留内存大小，防止被高位内存区域过度挤压占用
    setup_per_zone_lowmem_reserve();

#ifdef CONFIG_NUMA
    setup_min_unmapped_ratio();
    setup_min_slab_ratio();
#endif
    khugepaged_min_free_kbytes_update();
    return 0;
}
```

```
postcore_initcall(init_per_zone_wmark_min)
```

**calculate_min_free_kbytes()函数计算 min_free_kbytes:**

```
void calculate_min_free_kbytes(void)
{
    // 低位内存区域（除高端内存之外）的总和
    unsigned long lowmem_kbytes;

     // 待计算的 min_free_kbytes
    int new_min_free_kbytes;

    // 将低位内存区域内存容量总的页数转换为 KB
    //nr_free_buffer_pages(): ZONE_DMA 内存域和 ZONE_NORMAL 内存域中，高于
高水位线的空闲页数量。
    lowmem_kbytes = nr_free_buffer_pages() * (PAGE_SIZE >> 10);

    //min_free_kbytes 计算逻辑：对 lowmem_kbytes * 16 进行开平方
    new_min_free_kbytes = int_sqrt(lowmem_kbytes * 16);

    // min_free_kbytes 的范围限制在 128 到 262144 KB 之间
    //user_min_free_kbytes：用户设定的内核参数值
    if (new_min_free_kbytes > user_min_free_kbytes)
        min_free_kbytes = clamp(new_min_free_kbytes, 128, 262144);
    else
        pr_warn("min_free_kbytes is not updated to %d because user defined
value %d is preferred\n",
                new_min_free_kbytes, user_min_free_kbytes);
}
```

首先需要先计算出当前 NUMA 节点中所有低位内存区域（除高端内存之外）中内存总容量之和。也即是说 lowmem_kbytes 的值为：

**ZONE_DMA 区域中 managed_pages + ZONE_DMA32 区域中 managed_pages + ZONE_NORMAL 区域中 managed_pages** 。

**lowmem_kbytes 的计算逻辑在 nr_free_buffer_pages 方法中：**

```
unsigned long nr_free_buffer_pages(void)
{
    return nr_free_zone_pages(gfp_zone(GFP_USER));
}
```

**nr_free_zone_page 计算逻辑：**

```
/**
 * nr_free_zone_pages - count number of pages beyond high watermark
```

```
 * @offset: The zone index of the highest zone
 *
 * nr_free_zone_pages() counts the number of pages which are beyond the
 * high watermark within all zones at or below a given zone index.  For each
 * zone, the number of pages is calculated as:
 *
 *     nr_free_zone_pages = managed_pages - high_pages
 *
 * Return: number of pages beyond high watermark.
 */
static unsigned long nr_free_zone_pages(int offset)
{
    struct zoneref *z;
    struct zone *zone;

    unsigned long sum = 0;

    //获取当前 NUMA 节点中的所有物理内存区域 zone
    struct zonelist *zonelist = node_zonelist(numa_node_id(), GFP_KERNEL);

    // 计算所有物理内存区域内 managed_pages - high_pages 的总和
    for_each_zone_zonelist(zone, z, zonelist, offset) {
        unsigned long size = zone_managed_pages(zone);
        unsigned long high = high_wmark_pages(zone);
        if (size > high)
            sum += size - high;
    }
    return sum;//lowmem_kbytes 的值
}
```

nr_free_zone_pages 方法的计算逻辑本意是给定一个 zone index （方法参数 offset），计算范围为：**这个给定 zone 下面的所有低位内存区域**。

nr_free_zone_pages 方法会计算这些低位内存区域内在 high watermark 水位线之上的内存容量（ managed_pages - high_pages ）之和。作为该方法的返回值。**但此时我们正准备计算这些水位线，水位线还没有值**，所以此时这个方法的语义就是计算低位内存区域内被伙伴系统所管理的内存容量（ managed_pages ）之和。也就是我们想要的 lowmem_kbytes。

接下来在 init_per_zone_wmark_min 方法中会对 lowmem_kbytes * 16 进行开平方得到 new_min_free_kbytes。

$$\text{min_free_kbytes} = \sqrt{[\ \text{ZONE_DMA(managed_pages)} + \text{ZONE_DMA32(managed_pages)} + \text{ZONE_NORMAL(managed_pages)}\ ] * 16}$$

如果计算出的 new_min_free_kbytes 大于用户设置的内核参数值 user_min_free_kbytes，就设定为 clamp(new_min_free_kbytes, 128, 262144) ；如果小于用户设定的值就打印错误警告。

随后内核会根据这个 min_free_kbytes 在 setup_per_zone_wmarks() 方法中计算出该物理内存区域的三条水位线。

最后在 setup_per_zone_lowmem_reserve() 方法中计算内存区域的预留内存大小，防止被高位内存区域过度挤压占用。

### 4.5.2 计算物理内存区域的三条水位线

不考虑高端内存区域 ZONE_HIGHMEM。物理内存区域的三条水位线 WMARK_MIN，WMARK_LOW，WMARK_HIGH 的最终计算逻辑是在__setup_per_zone_wmarks 函数中。

```
void setup_per_zone_wmarks(void)
{
    struct zone *zone;
    static DEFINE_SPINLOCK(lock);

    spin_lock(&lock);
    __setup_per_zone_wmarks();
    spin_unlock(&lock);
    for_each_zone(zone)
        zone_pcp_update(zone, 0);
}
```

```
static void __setup_per_zone_wmarks(void)
{
   //将 min_free_kbytes 转换为页
    unsigned long pages_min = min_free_kbytes >> (PAGE_SHIFT - 10);
 // 所有低位内存区域 managed_pages 之和
    unsigned long lowmem_pages = 0;
    struct zone *zone;
    unsigned long flags;

    /* Calculate total number of !ZONE_HIGHMEM and !ZONE_MOVABLE pages */
    for_each_zone(zone) {
        if (!is_highmem(zone) && zone_idx(zone) != ZONE_MOVABLE)
            lowmem_pages += zone_managed_pages(zone);
    }

    // 循环计算各个内存区域中的水位线
    for_each_zone(zone) {
        u64 tmp;
        spin_lock_irqsave(&zone->lock, flags);
        tmp = (u64)pages_min * zone_managed_pages(zone);
         // 计算 WMARK_MIN 水位线的核心方法。按比例分配全局最小预留页
        do_div(tmp, lowmem_pages);
```

```
        if (is_highmem(zone) || zone_idx(zone) == ZONE_MOVABLE) {
            unsigned long min_pages;
            min_pages = zone_managed_pages(zone) / 1024;
            min_pages = clamp(min_pages, SWAP_CLUSTER_MAX, 128UL);
            zone->_watermark[WMARK_MIN] = min_pages;
        } else {
               // WMARK_MIN 水位线
               zone->_watermark[WMARK_MIN] = tmp;
        }
        tmp = max_t(u64, tmp >> 2,
                mult_frac(zone_managed_pages(zone),
                      watermark_scale_factor, 10000));
        zone->watermark_boost = 0;
        zone->_watermark[WMARK_LOW]  = min_wmark_pages(zone) + tmp;
        zone->_watermark[WMARK_HIGH] = low_wmark_pages(zone) + tmp;
        zone->_watermark[WMARK_PROMO] = high_wmark_pages(zone) + tmp;
        spin_unlock_irqrestore(&zone->lock, flags);
    }
    /* update totalreserve_pages */
    calculate_totalreserve_pages();
}
```

在 for_each_zone 循环内依次遍历 NUMA 节点中的所有内存区域 zone，计算每个内存区域 zone 里的内存水位线。核心逻辑是这两句：

```
        tmp = (u64)pages_min * zone_managed_pages(zone);
        do_div(tmp, lowmem_pages);
```

**lowmem_pages** 是所有低位内存区域 managed_pages 之和
**pages_min** 是 系统对核心内存域的总预留要求
两句表达的含义是：
全局总预留页 * (当前 zone 容量 / 所有低位内存区域 managed_pages 之和) 。按 当前 zone 在所有低位内存区域 managed_pages 之和中的占比 来分配全局预留页。

比如：当前 NUMA 节点中有两个 zone ：ZONE_DMA 和 ZONE_NORMAL，内存容量大小分别是：100 M 和 800 M。那么 ZONE_DMA 与 ZONE_NORMAL 之间的比例就是 1 ：8。根据这个比例，
ZONE_DMA 区域里的 WMARK_MIN 水位线就是：min_free_kbytes * 1 / 8 。
ZONE_NORMAL 区域里的 WMARK_MIN 水位线就是：min_free_kbytes * 7 / 8 。

再根据 WMARK_MIN 的计算 WMARK_LOW， WMARK_HIGH 的值，WMARK_LOW 的值是 WMARK_MIN 的 1.25 倍，WMARK_HIGH 的值是 WMARK_LOW 的 1.5 倍。

```
   tmp = max_t(u64, tmp >> 2,
                mult_frac(zone_managed_pages(zone),
                      watermark_scale_factor, 10000));
```

表示：通过内核参数 watermark_scale_factor 来调节水位线：WMARK_MIN，WMARK_LOW，WMARK_HIGH 之间的间距。

## 4.5.3 watermark_scale_factor 调整水位线的间距

为了避免内核的直接内存回收 direct reclaim 阻塞进程影响系统的性能，所以我们需要尽量保持内存区域中的剩余内存容量尽量在 WMARK_MIN 水位线之上，但是有一些极端情况，比如突然遇到网络流量增大，需要短时间内申请大量的内存来存放网络请求数据，此时 kswapd 回收内存的速度可能赶不上内存分配的速度，从而造成直接内存回收 direct reclaim，影响系统性能。

在内存分配过程中，剩余内存容量处于 WMARK_MIN 与 WMARK_LOW 水位线之间会唤醒 kswapd 进程来回收内存，直到内存容量恢复到 WMARK_HIGH 水位线之上。剩余内存容量低于 WMARK_MIN 水位线时就会**触发直接内存回收 direct reclaim**。

剩余内存容量高于 WMARK_LOW 水位线又不会唤醒 kswapd 进程，因此 kswapd 进程活动的关键范围在 WMARK_MIN 与 WMARK_LOW 之间，而为了应对这种突发的网络流量暴增，我们需要保证 kswapd 进程活动的范围大一些，这样内核就能够时刻进行内存回收使得剩余内存容量较长时间的保持在 WMARK_HIGH 水位线之上。

**这样就要求 WMARK_MIN 与 WMARK_LOW 水位线之间的间距不能太小**，因为 WMARK_LOW 水位线之上就不会唤醒 kswapd 进程了。

内核引入 /proc/sys/vm/watermark_scale_factor 参数来调节水位线之间的间距。该内核参数默认值为 10，最大值为 3000。

水位线间距计算公式：(watermark_scale_factor / 10000) * managed_pages 。

内核中水位线间距计算逻辑是：(WMARK_MIN / 4) 与 (zone_managed_pages * watermark_scale_factor / 10000) 之间较大的那个值。用户动态调整 watermark_scale_factor 参数，内核会动态重新计算水位线之间的间距，使得 WMARK_MIN 与 WMARK_LOW 之间留有足够的缓冲余地，使得 kswapd 能够有时间回收足够的内存，从而解决直接内存回收

导致的**性能抖动**问题。

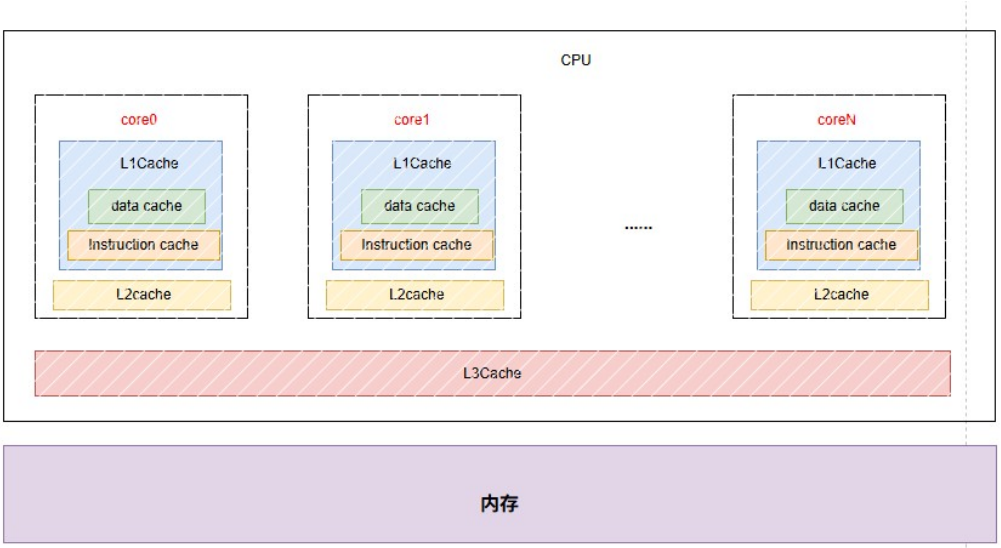
## 4.6 物理内存区域中的冷热页

寄存器是离 CPU 最近的，CPU 在访问寄存器的时候速度近乎于 0 个时钟周期，访问速度最快，基本没有时延。而访问内存则需要 50 - 200 个时钟周期。所以为了弥补 CPU 与内存之间巨大的速度差异，提高 CPU 的处理效率和吞吐，于是我们引入了 L1 , L2 , L3 高速缓存集成到 CPU 中。CPU 访问高速缓存仅需要用到 1 - 30 个时钟周期，CPU 中的高速缓存是对内存热点数据的一个缓存。

**热页**就是已经加载进 CPU 高速缓存中的物理内存页.

**冷页**就是还未加载进 CPU 高速缓存中的物理内存页，

冷页是热页的后备选项。




CPU 访问高速缓存的速度比访问内存的速度快大约 10 倍，引入高速缓存的目的在于消除 CPU 与内存之间的速度差距，CPU 用高速缓存来用来存放内存中的热点数据。

根据程序的时间局部性原理可以知道，内存的数据一旦被访问，那么它很有可能在短期内被再次访问，如果我们把经常访问的物理内存页缓存在 CPU 的高速缓存中，那么当进程再次访问的时候就会直接命中 CPU 的高速缓存，避免了进一步对内存的访问，极大提升了应用程序的性能。

**程序局部性原理表现为：**时间局部性和空间局部性。

**时间局部性**是指如果程序中的某条指令一旦执行，则不久之后该指令可能再次被执行；如果某块数据被访问，则不久之后该数据可能再次被访问。

**空间局部性**是指一旦程序访问了某个存储单元，则不久之后，其附近的存储单元也将被访问。

物理内存区域 struct zone 包含了一个 struct per_cpu_pages 类型的数组 per_cpu_pageset。其中内核关于冷热页的管理全部封装在 struct per_cpu_pageset 结构中. 内核为每个 CPU 单独分配一个 per_cpu_pages 实例，用于缓存空闲页帧。

```
struct zone {
```

```
        struct per_cpu_pages    __percpu *per_cpu_pageset;

}
```

因为每个 CPU 都有自己独立的高速缓存，所以每个 CPU 对应一个 per_cpu_pageset 结构，per_cpu_pageset 指向一块连续的虚拟内存地址， 内存块的大小是，cpu。

在 NUMA 内存架构下，每个物理内存区域都是属于一个特定的 NUMA 节点，NUMA 节点中包含了一个或者多个 CPU，NUMA 节点中的每个内存区域会关联到一个特定的 CPU 上，但 struct zone 结构中的 per_cpu_pages 包含的是系统中所有 CPU 的高速缓存页。

虽然一个内存区域关联到了 NUMA 节点中的一个特定 CPU 上，但是其他 CPU 依然可以访问该内存区域中的物理内存页，因此其他 CPU 上的高速缓存仍然可以包含该内存区域中的物理内存页。

每个 CPU 都可以访问系统中的所有物理内存页，尽管访问速度不同（由于 NUMA 架构的原因），因此特定的物理内存区域 struct zone 不仅要考虑到所属 NUMA 节点中相关的 CPU，还需要照顾到系统中的其他 CPU。

```
struct per_cpu_pages {
    spinlock_t lock;    /* Protects lists field */
    int count;      /* number of pages in the list */
    int high;       /* high watermark, emptying needed */
    int high_min;       /* min high watermark */
    int high_max;       /* max high watermark */
    int batch;      /* chunk size for buddy add/remove */
    u8 flags;       /* protected by pcp->lock */
    u8 alloc_factor;    /* batch scaling factor during allocate */
#ifdef CONFIG_NUMA
    u8 expire;      /* When 0, remote pagesets are drained */
#endif
    short free_count;   /* consecutive free count */

    /* Lists of pages, one per migrate type stored on the pcp-lists */
    struct list_head lists[NR_PCP_LISTS];
} ____cacheline_aligned_in_smp;
```

**int count** ：表示集合中包含的物理页数量，如果该结构是热页集合，则表示加载进 CPU 高速缓存中的物理页面个数。

**struct list_head lists：**该 list 是一个双向链表，保存了当前 CPU 的热页或者冷页。该列表按“迁移类型”分类（可迁移，可回收，不可迁移）。每种迁移类型对应一条独立链表，确保分配时能快速获取对应类型的页，避免内存碎片化。

**int batch：**cpu 高速缓存从伙伴系统批量获取或者批量释放给伙伴系统的物理页面个数

**int high：**如果集合中页面的数量 count 值超过了 high 的值，那么表示 list 中的页面太多了，内核会从高速缓存中释放 batch 个页面到物理内存区域中的伙伴系统中。

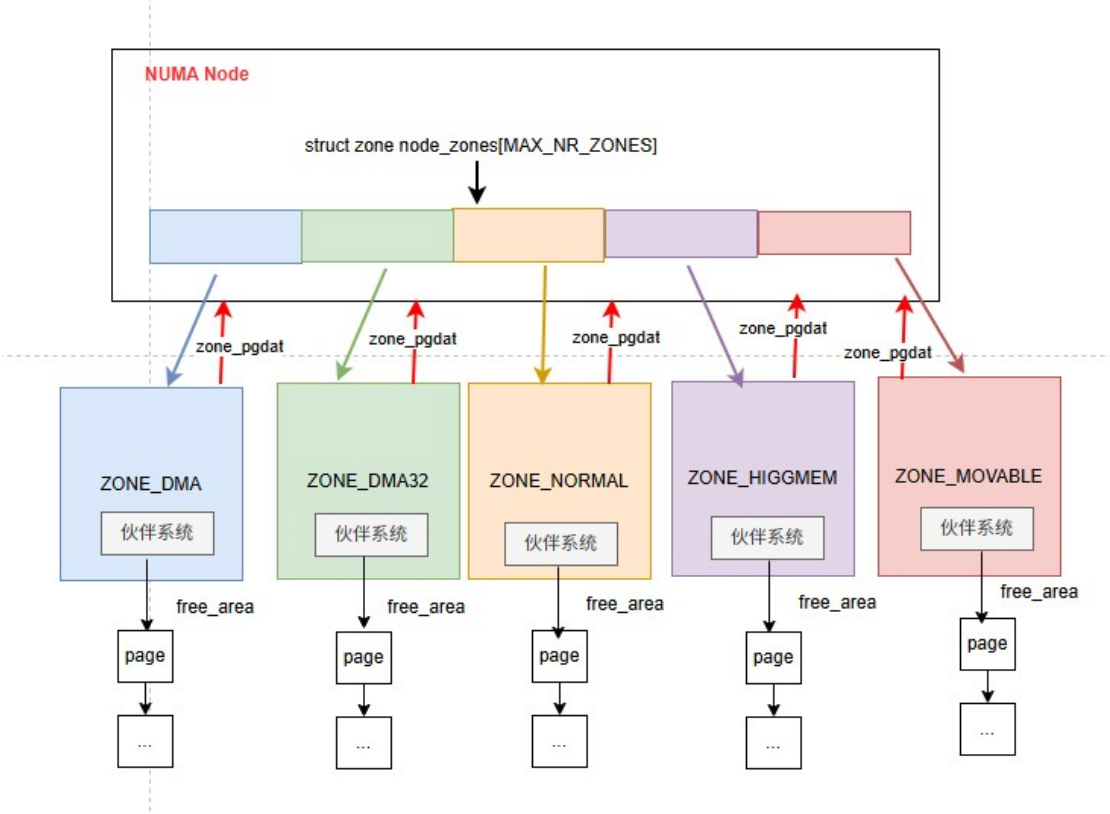
per_cpu_pageset 直接管理当前 zone 中每个 CPU 的本地热页缓存。

**冷热页关联逻辑：**

热页留存：CPU 频繁分配 / 释放的页优先存入自身的 per_cpu_pages 实例（lists 链表），避免进入全局伙伴系统，保持本地访问（热页属性）；

冷页过滤：当缓存数量超过 high 水位线时，批量释放「次热 / 冷却」的页到全局 LRU 链表，完成热页→冷页的降级；

## 4.7 内核中描述物理内存页




NUMA 架构下内存被划分成了一个一个的内存节点（NUMA Node），在每个 NUMA 节点中，内核又根据节点内物理内存的功能用途不同，将 NUMA 节点内的物理内存划分为四个物理内存区域分别是：ZONE_DMA，ZONE_DMA32，ZONE_NORMAL，ZONE_HIGHMEM。其中 ZONE_MOVABLE 区域是逻辑上的划分，主要是为了防止内存碎片和支持内存的热插拔。

物理内存区域中管理的就是物理内存页（ Linux 内存管理的最小单位），内核对物理内存的换入，换出，回收，内存映射等操作的单位就是页。**内核为每一个物理内存区域分配了一个伙伴系统**，用于管理该物理内存区域下所有物理内存页面的分配和释放。

Linux 默认支持的物理内存页大小为 4KB，在 64 位体系结构中还可以支持 8KB，有的处理器还可以支持 4MB，支持物理地址扩展 PAE 机制的处理器上还可以支持 2MB。

**Linux 会默认采用 4KB 作为标准物理内存页的大小的原因：**

首先关于物理页面的大小，Linux 规定必须是 2 的整数次幂，因为 2 的整数次幂可以将一些数学运算转换为移位操作，比如乘除运算可以通过移位操作来实现，这样效率更高。

在内存紧张的时候，内核会将不经常使用到的物理页面进行换入换出等操作，还有在内

存与文件映射的场景下，都会涉及到与磁盘的交互，数据在磁盘中组织形式也是根据一个磁盘块一个磁盘块来管理的，4kB 和 4MB 都是磁盘块大小的整数倍，但在大多数情况下，内存与磁盘之间传输小块数据时会更加的高效，所以综上所述内核会采用 4KB 作为默认物理内存页大小。

假设我们有 4G 大小的物理内存，每个物理内存页大小为 4K，那么这 4G 的物理内存会被内核划分为 1M 个物理内存页，内核使用一个 struct page 的结构体来描述物理内存页，而每个 struct page 结构体占用内存大小为 40 字节，那么内核就需要用额外的 40 * 1M = 40M 的内存大小来描述物理内存页。

对于 4G 的物理内存来说，内核就需要使用 1M 个物理页面来管理，1M 个物理页的数量已经是非常庞大的了，因此在后续的内核迭代中，对于 struct page 结构的任何微小改动，都可能导致用于管理物理内存页的 struct page 实例所需要的内存暴涨。

对于 4G 的物理内存来说，内核就需要使用 1M 个物理页面来管理，1M 个物理页的数量已经是非常庞大的了，因此在后续的内核迭代中，对于 struct page 结构的任何微小改动，都可能导致用于管理物理内存页的 struct page 实例所需要的内存暴涨。

```
struct page {
    //页状态标志位
    unsigned long flags;
    union {
        struct { //场景 1：页缓存（Page Cache）与匿名页
            union {
                //用来指向物理页 page 被放置在了哪个 lru 链表上
                struct list_head lru;

                /* Or, for the Unevictable "LRU list" slot */
                struct { // 不可换出页
                    /* Always even, to negate PageTail */
                    void *__filler;
                    /* Count page's or folio's mlocks */
                    unsigned int mlock_count;
                };
                /* Or, free page */
                struct list_head buddy_list;// 伙伴系统空闲页链表
                struct list_head pcp_list;// PCP 每 CPU 缓存链表
            };


          //如果 page 为文件页的话，低位为 0，指向 page 所在的 page cache
           // 如果 page 为匿名页的话，低位为 1，指向其对应虚拟地址空间的匿名映射区 anon_vma
            struct address_space *mapping;
            union {
                // 如果 page 为文件页的话，index 为 page 在 page cache 中的索引
```

```
                // 如果 page 为匿名页的话，表示匿名页在对应进程虚拟内存区域
                 VMA 中的偏移
                pgoff_t index;
                unsigned long share;    /* share count for fsdax */
            };
            unsigned long private;
        };
        struct { //场景 2: 网络页池（page_pool，netstack 用）
             unsigned long pp_magic;
            struct page_pool *pp;
            unsigned long _pp_mapping_pad;
            unsigned long dma_addr;
            atomic_long_t pp_ref_count;
        };
        struct { //场景 3: 复合页
            unsigned long compound_head;    /* Bit zero is set */
        };
        struct { // 场景 4: 设备页
            /** @pgmap: Points to the hosting device page map. */
            struct dev_pagemap *pgmap;
            void *zone_device_data;
        };
         struct rcu_head rcu_head;
    };
    //复用字段：映射计数或页类型
    union {
        // 内核中引用该物理页的次数，表示该物理页的活跃程度。
        atomic_t _mapcount;
        unsigned int page_type;//标记页的具体用途
    };
    /* Usage count. *DO NOT USE DIRECTLY*. See page_ref.h */
    atomic_t _refcount;
#ifdef CONFIG_MEMCG
    unsigned long memcg_data;
#endif

#if defined(WANT_PAGE_VIRTUAL)
    void *virtual;  // 内存页对应的虚拟内存地址
#endif /* WANT_PAGE_VIRTUAL */
#ifdef LAST_CPUPID_NOT_IN_PAGE_FLAGS
    int _last_cpupid;
#endif
#ifdef CONFIG_KMSAN
    struct page *kmsan_shadow;
```

```
    struct page *kmsan_origin;
#endif
} _struct_page_alignment;
```

struct page 结构在不同场景下的使用方式：

第一种使用方式是内核直接分配使用一整页的物理内存，内核中的物理内存页有两种类型，分别用于不同的场景：

**匿名页（Anonymous Pages）**：没有文件背景的页面，如进程的堆栈、数据段等。这些页面在内存紧张时会被交换到 swap 分区。

**文件页（File Pages）**：文件页中的数据来自于磁盘中的文件，文件页需要先关联一个磁盘中的文件，然后再和进程虚拟地址空间建立映射供进程使用，使得进程可以通过操作虚拟内存实现对文件的操作，这就是我们常说的内存文件映射。如果页面是干净的（与磁盘一致），则可以直接丢弃；如果是脏的，则需要写回磁盘后才能回收。

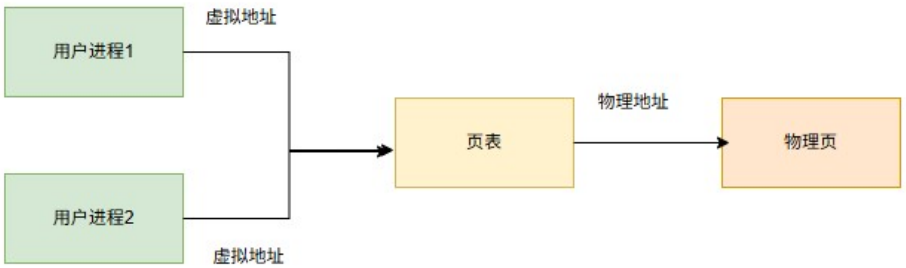
**struct page 中的 struct address_space *mapping;** 字段用来指示当前物理内存页 是一个匿名页还是一个文件页。

如果当前物理内存页 struct page 是一个文件页的话，那么 mapping 指针的最低位会被设置为 0 ，指向该内存页关联文件的 struct address_space（页高速缓存），**pgoff_t index** 字段表示该内存页 page 在页高速缓存 page cache 中的 index 索引。内核会利用这个 index 字段从 page cache 中查找该物理内存页。同时该 pgoff_t index 字段也表示该内存页中的文件数据在文件内部的偏移 offset。偏移单位为 page size。

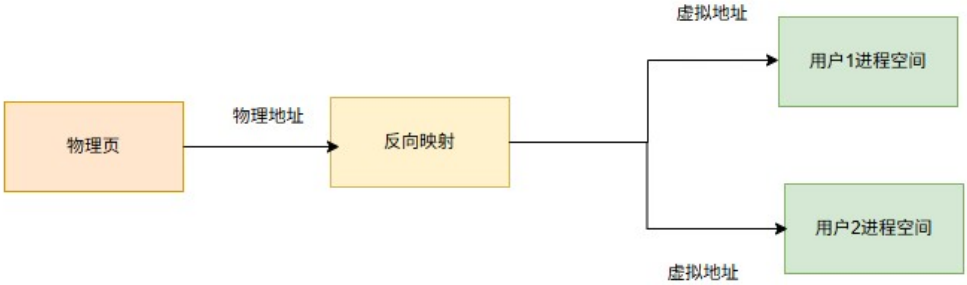
如果当前物理内存页 struct page 是一个匿名页的话，那么 mapping 指针的最低位会被设置为 1 ，指向该匿名页在进程虚拟内存空间中的匿名映射区域 struct anon_vma 结构（每个匿名页对应唯一的 anon_vma 结构），用于物理内存到虚拟内存的反向映射。

## 4.8 匿名页的反向映射

通常所说的内存映射是正向映射，即从虚拟内存到物理内存的映射。




反向映射则是从物理内存到虚拟内存的映射。使用反向映射机制找到映射该物理内存页的所有进程虚拟地址空间 VMA 。用于当某个物理内存页需要进行回收或迁移时，此时需要去找到这个物理页被映射到了哪些进程的虚拟地址空间中，并断开它们之间的映射。




## 4.8.1 反向映射中使用到的数据结构

主要使用三个数据结构：

1、struct page;

2、struct vm_area_struct;

3、struct anon_vma

4、struct anon_vma_chain

1、物理页面描述符 **struct page** 中与反向映射有关的两个关键成员是 mapping 和 __mapcount：

```
struct page {
            struct address_space *mapping;
            union {
                pgoff_t index;      /* Our offset within mapping. */
                unsigned long share;    /* share count for fsdax */
           };

}
```

mapping：字段 mapping 用于区分匿名页面和基于文件映射的页面，如果该字段的最低位被置位了，那么该字段包含的是指向 anon_vma 结构（用于匿名页面）的指针；否则，该字段包含指向 address_space 结构的指针（用于基于文件映射的页面）。

index 成员指向了该 page 在整个 vm_area_struct 中的偏移。

__mapcount：_mapcount 表示共享该物理页面的页表现数目，即有多少个进程页表的 pte 映射到该物理页面。该值初始值为-1，每增减一个 pte 映射该值+1

即该 page 映射了多少个进程的虚拟内存空间，一个 page 可以被多个进程映射。

反向映射中主要使用到的三个重要的数据结构：

**2、struct vm_area_struct，（VMA）用于描述进程地址空间的一段区域。其中，与匿名页反向映射相关的字段属性：**

```
struct vm_area_struct {
        ......
```

```
        // 链表头：所有关联的 AVC 节点都挂在这个链表上
        struct list_head anon_vma_chain;
        //专属 anon_vma（如 COW 后子进程的专属 anon_vma）
        struct anon_vma *anon_vma;
        ......
}
```

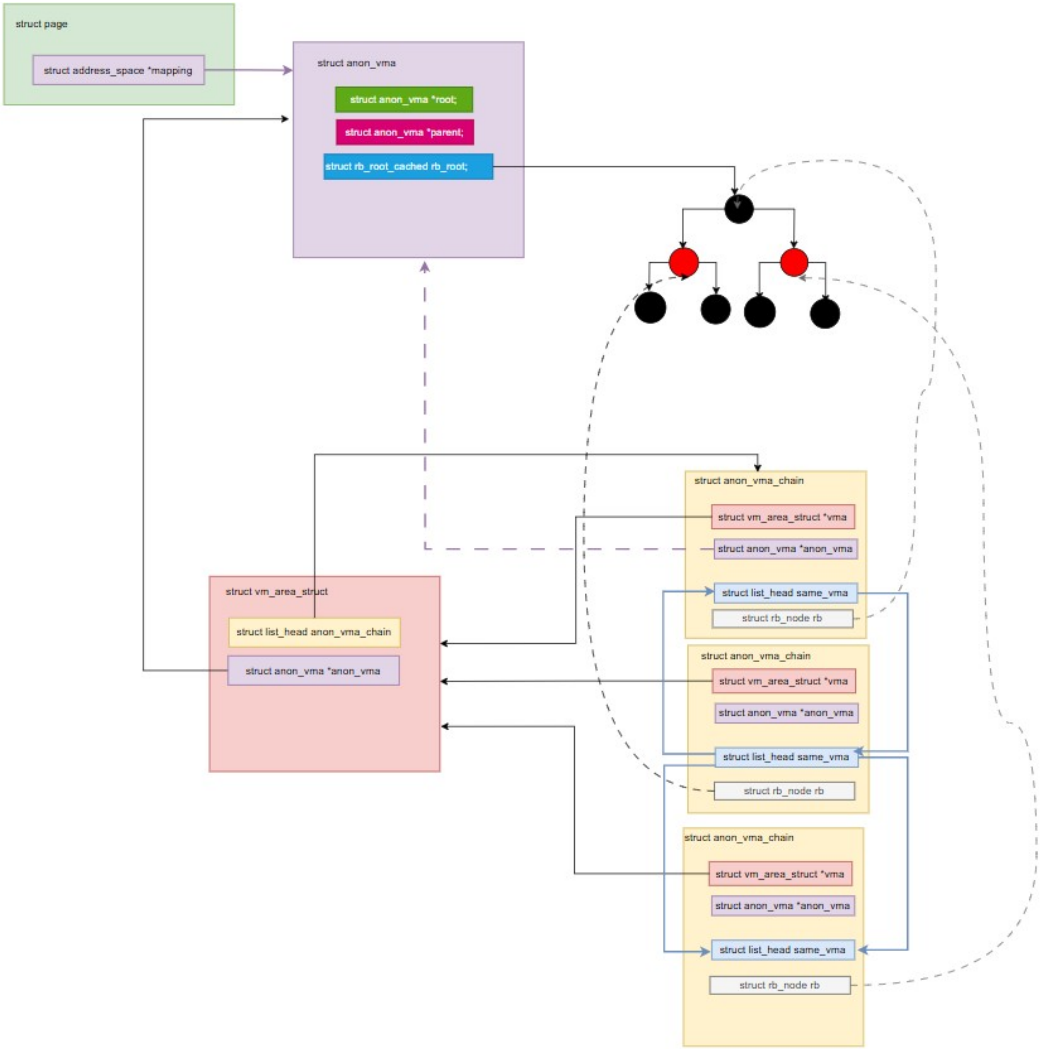
**3、struct anon_vma，(AV)用于连接物理页面对应的 struct page 和 VMA 的 vm_area_struct。**

```
struct anon_vma {
    /*
    无论当前 anon_vma 是树的“根节点”“子节点”还是“孙节点”，其 root 字段永远指向 整个 anon_vma 树的最顶层始祖节点（即“树根”）；
    */
    struct anon_vma *root;      ///指向 anon_vma 数据结构的根节点
    struct rw_semaphore rwsem;  //保护 anon_vma 数据结构中链表的读写信号量
    atomic_t refcount;// 引用计数
    unsigned long num_children;
    /* Count of VMAs whose ->anon_vma pointer points to this object. */
    unsigned long num_active_vmas;
    struct anon_vma *parent;    // 指向父 anon_vma 数据结构
   // 红黑树根节点。anon_vma 内部有一颗红黑树管理的全部都是与该 anon_vma 关联的 anon_vma_chain。
    struct rb_root_cached rb_root;
};
```

**4、struct anon_vma_chain，(AVC)起枢纽的作用，比如连接父子进程间的 struct anon_vma**

```
    struct anon_vma_chain {
     //  指向关联的 VMA
    struct vm_area_struct *vma;
   // 指向关联的 anon_vma
    struct anon_vma *anon_vma;
    // 链表节点：串联同一 VMA 的所有 AVC
    struct list_head same_vma;
    // 红黑树节点：把 anon_vma_chain 添加到 anon_vma->rb_root 的红黑树中
    struct rb_node rb;
    unsigned long rb_subtree_last;
#ifdef CONFIG_DEBUG_VM_RB
    unsigned long cached_vma_start, cached_vma_last;
#endif
};
```

sruct anon_vma 、 struct anon_vma_chain、 struct page、与 vm_area_struct 整体关系流程图：




## 4.8.2 匿名反向映射过程

### 4.8.2.1 sruct anon_vma 、struct anon_vma_chain 和 struct page 三者之间建立联系

内核利用 sruct anon_vma 结构和 struct anon_vma_chain 结构体完成匿名内存页的反向映射。进程利用 fork 系统调用创建子进程的时候，内核会将父进程的虚拟内存空间相关的内容拷贝到子进程的虚拟内存空间中，此时子进程的虚拟内存空间和父进程的虚拟内存空间是一模一样的，其中虚拟内存空间中映射的物理内存页也是一样的，在内核中都是同一份，在父进程和子进程之间共享（包括 anon_vma 和 anon_vma_chain）。

当进程开始访问这段虚拟内存时，内核会产生缺页中断，在缺页中断处理函数中才会去真正的分配物理内存（这时才会为子进程创建自己的 anon_vma 和 anon_vma_chain），并建立虚拟内存与物理内存之间的映射关系（正向映射）。

```
static vm_fault_t handle_pte_fault(struct vm_fault *vmf)
{
    ......
    if (!vmf->pte)
        //处理匿名页缺页和文件缺页
        return do_pte_missing(vmf);
    if (!pte_present(vmf->orig_pte))
       //子进程缺页处理
        return do_swap_page(vmf);
    ......
unlock:
    pte_unmap_unlock(vmf->pte, vmf->ptl);
    return 0;
}
```

```
static vm_fault_t do_pte_missing(struct vm_fault *vmf)
{
    if (vma_is_anonymous(vmf->vma))
          //处理匿名缺页
        return do_anonymous_page(vmf);
    else
        //处理文件缺页
       return do_fault(vmf);
}
```

内核在 do_anonymous_page 函数完成了中 struct anon_vma 结构和 struct anon_vma_chain 结构的创建以及相关匿名页反向映射数据结构的相互关联。

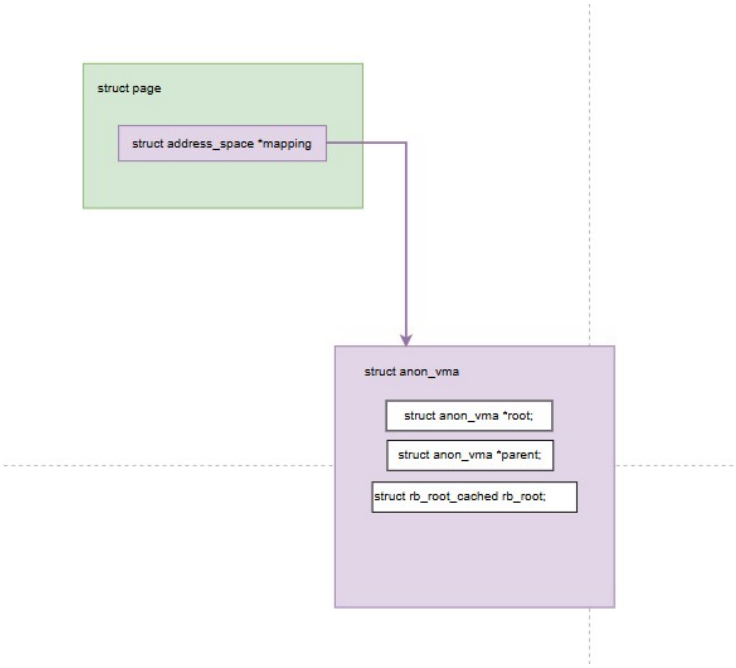
```
static vm_fault_t do_anonymous_page(struct vm_fault *vmf)
{
    bool uffd_wp = vmf_orig_pte_uffd_wp(vmf);
    struct vm_area_struct *vma = vmf->vma;
    unsigned long addr = vmf->address;
    struct folio *folio;
    vm_fault_t ret = 0;
    int nr_pages = 1;
    pte_t entry;
    int i;
    ......省略虚拟内存到物理内存正向映射相关逻辑......

    //准备匿名 VMA
    if (unlikely(anon_vma_prepare(vma)))
        goto oom;
```

```
    //分配匿名页组
    folio = alloc_anon_folio(vmf);
    if (IS_ERR(folio))
        return 0;
    if (!folio)
        goto oom;
     ......
     // 建立反向映射关系
    folio_add_new_anon_rmap(folio, vma, addr);
 ........省略虚拟内存到物理内存正向映射相关逻辑.........
}
```

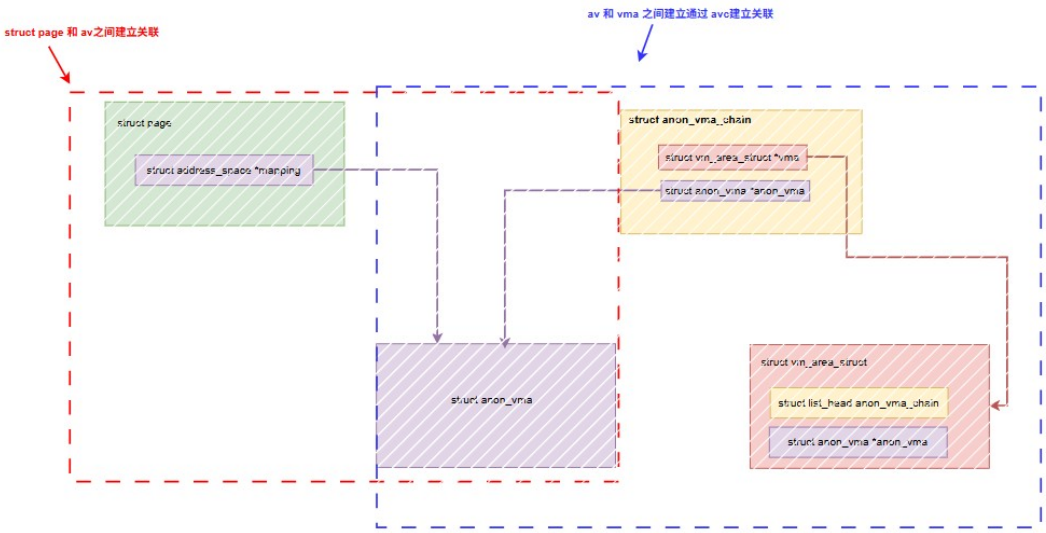
在 do_anonymous_page 匿名页缺页处理函数中会为 struct vm_area_struct 结构创建匿名页相关的 struct anon_vma 结构和 struct anon_vma_chain 结构。并在 anon_vma_prepare 函数中**实现 anon_vma 和 anon_vma_chain 之间的关联**，调用 alloc_anon_folio 分配匿名页。并在 folio_add_new_anon_rmap 函数中完成 struct page 与 anon_vma 的关联。

#### 4.8.2.2 struct page 与进程虚拟内存空间 struct vm_area_struct 之间建立联系

匿名页的反向映射关键就是建立物理内存页 struct page 与进程虚拟内存空间 VMA 之间的映射关系。匿名页的 struct page 中的 mapping 指针指向的是 struct anon_vma 结构。




只要实现了 anon_vma 与 vm_area_struct 之间的关联，page 到 vm_area_struct 之间的映射就建立起来了，struct anon_vma_chain 结构将 anon_vma 与 vm_area_struct 建立关系。



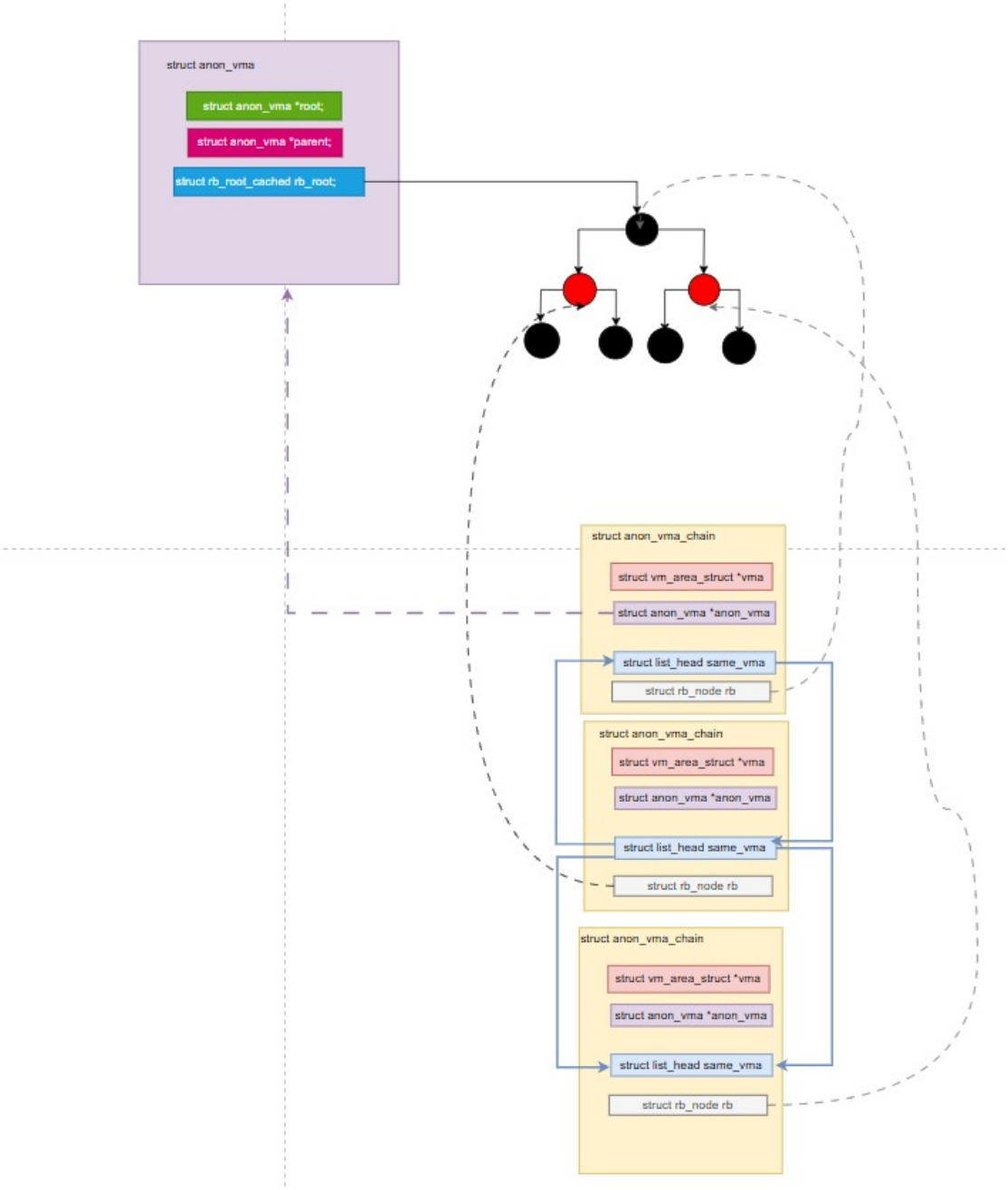

匿名页 struct page 与 anon_vma 存在对应关系，anstruct anon_vma_chain 结构通过其中的 vma 指针和 anon_vma 指针将相关的匿名页与其映射的进程虚拟内存空间关联了起来。因此可以通过 anon_vma 找到 anon_vma_chain ，进而找到 vm_area_struct。

### 4.8.2.3 sruct anon_vma 、 struct anon_vma_chain、 struct page、与 vm_area_struct 整体关系

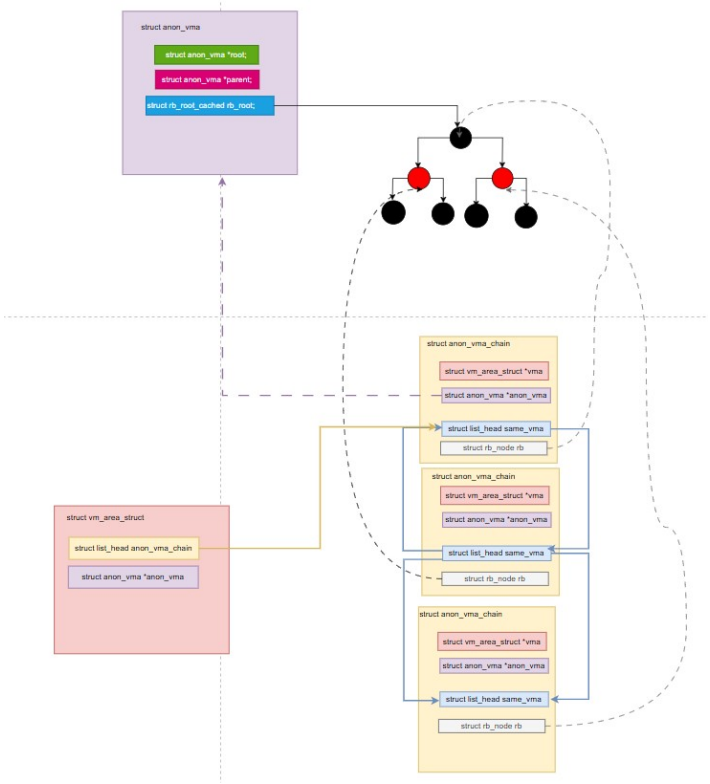
每个匿名页对应唯一的 anon_vma 结构，但是一个匿名物理页可以映射到不同进程的虚拟内存空间中，每个进程的虚拟内存空间都是独立的，也就是说不同的进程就会有不同的 VMA。不同的 VMA 意味着同一个匿名页 anon_vma 就会对应多个 anon_vma_chain。

struct anon_vma 结构中使用 rb_root 字段管理了一颗红黑树，这颗红黑树上管理的全部都是与该 anon_vma 关联的 anon_vma_chain。我们可以通过 struct page 中的 mapping 指针找到 anon_vma，然后遍历 anon_vma 中的这颗红黑树 rb_root ，从而找到与其关联的所有 anon_vma_chain。

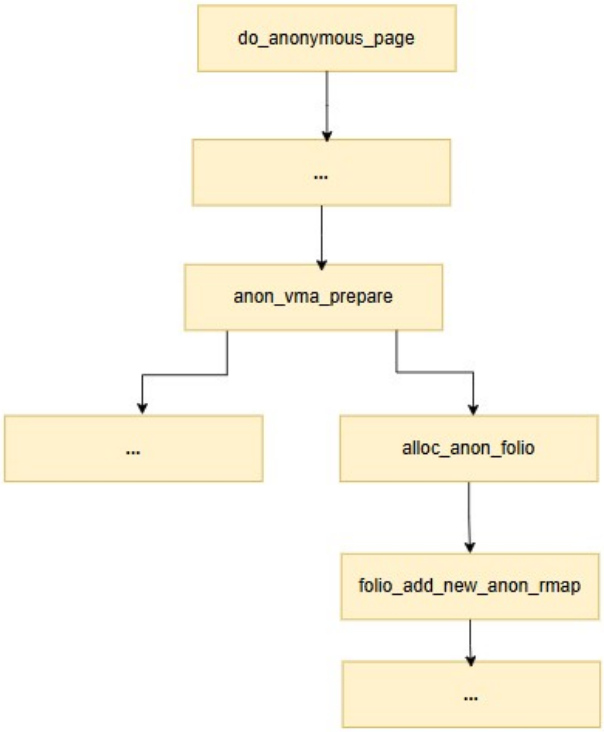
struct anon_vma_chain 结构中的 rb 字段表示 anon_vma_chain 在对应 anon_vma 管理的红黑树中的节点。

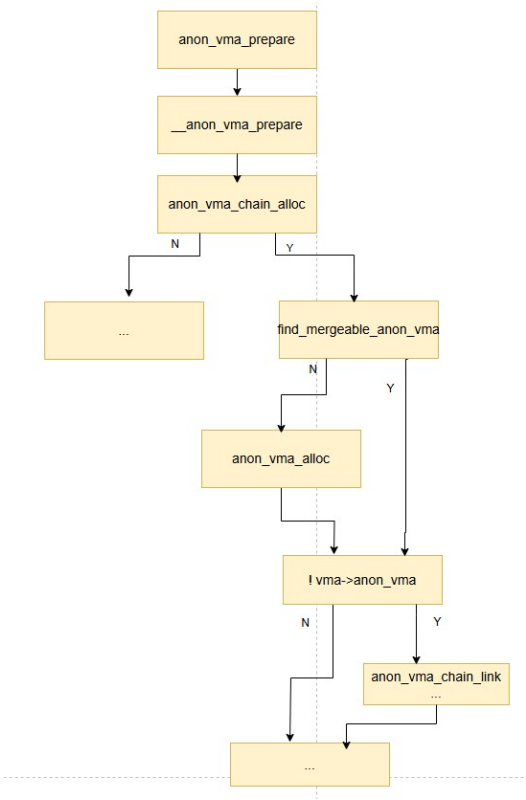



vm_area_struct 表示的是进程虚拟内存空间中的一段虚拟内存区域，这块虚拟内存区域中可能会包含多个匿名页，所以 VMA 与物理内存页 page 也是有一对多的映射关系存在。struct anon_vma_chain 结构中有一个列表结构 same_vma，列表 same_vma 中存储的 anon_vma_chain 对应的 VMA 全都是一样的，而列表元素 anon_vma_chain 中的 anon_vma 却是不一样的，内核用链表结构 same_vma 存储进程对应虚拟内存区域 VMA 中所包含的所有匿名页。struct vm_area_struct 结构中的 struct list_head anon_vma_chain 指向的也是这个列表 same_vma，图中黄色箭头。




sruct anon_vma 、 struct anon_vma_chain、 struct page、与 vm_area_struct 整体关系的函数处理流程:






在 do_anonymous_page 中首先会调用

**anon_vma_prepare** ：方法来为匿名页创建 anon_vma 实例和 anon_vma_chain 实例，并建立它们之间的关联关系。

```
int __anon_vma_prepare(struct vm_area_struct *vma)
{
    // 获取进程虚拟内存空间
    struct mm_struct *mm = vma->vm_mm;
    // 准备为匿名页分配 anon_vma 以及 anon_vma_chain
    struct anon_vma *anon_vma, *allocated;
    struct anon_vma_chain *avc;

    might_sleep();
    // 分配 anon_vma_chain 实例
    avc = anon_vma_chain_alloc(GFP_KERNEL);
    if (!avc)
        goto out_enomem;
    // 在相邻的虚拟内存区域 VMA 中查找可复用的 anon_vma
    anon_vma = find_mergeable_anon_vma(vma);
    allocated = NULL;
    if (!anon_vma) {
         // 没有可复用的 anon_vma 则创建一个新的实例
        anon_vma = anon_vma_alloc();
        if (unlikely(!anon_vma))
```

```
            goto out_enomem_free_avc;
        anon_vma->num_children++; /* self-parent link for new root */
        allocated = anon_vma;
    }
    anon_vma_lock_write(anon_vma);
    /* page_table_lock to protect against threads */
    spin_lock(&mm->page_table_lock);
    if (likely(!vma->anon_vma)) {
        // VMA 中的 anon_vma 属性就是在这里赋值的
        vma->anon_vma = anon_vma;
        // 建立反向映射关联
        anon_vma_chain_link(vma, avc, anon_vma);
        anon_vma->num_active_vmas++;
        allocated = NULL;
        avc = NULL;
    }
  .......省略......
}
```

**然后，__anon_vma_prepare 函数中再调用 anon_vma_chain_link** ：建立 anon_vma，anon_vma_chain，vm_area_struct 三者的联系。

```
static void anon_vma_chain_link(struct vm_area_struct *vma,
                struct anon_vma_chain *avc,
                struct anon_vma *anon_vma)
{
    // 通过 anon_vma_chain 关联 anon_vma 和对应的 vm_area_struct
    avc->vma = vma;
    avc->anon_vma = anon_vma;
    // 将 anon_vma_chain 中的 same_vma 链表加入到 vm_area_struct 中
的 anon_vma_chain 链表 anon_vma_chain 中
    list_add(&avc->same_vma, &vma->anon_vma_chain);
    // 将初始化好的 anon_vma_chain 加入到 anon_vma 管理的红黑树 rb_root 中
    anon_vma_interval_tree_insert(avc, &anon_vma->rb_root);
}
```

**anon_vma_prepare 函数处理完成之后 anon_vma、anon_vma_chain\vm_area_struct 已经建立了联系。 然后使用 alloc_anon_folio** 函数从伙伴系统中申请匿名页。当获取到 page 实例之后，通过 **folio_add_new_anon_rmap 函数**最终建立起 page 到 vm_area_struct 的整条反向映射链路。

```
void folio_add_new_anon_rmap(struct folio *folio, struct vm_area_struct
*vma,
        unsigned long address)
{
```

```
    int nr = folio_nr_pages(folio);

    VM_WARN_ON_FOLIO(folio_test_hugetlb(folio), folio);
    VM_BUG_ON_VMA(address < vma->vm_start ||
            address + (nr << PAGE_SHIFT) > vma->vm_end, vma);
    __folio_set_swapbacked(folio);
    __folio_set_anon(folio, vma, address, true);
    if (likely(!folio_test_large(folio))) {
        /* increment count (starts at -1) */
        atomic_set(&folio->_mapcount, 0);
        SetPageAnonExclusive(&folio->page);
    } else if (!folio_test_pmd_mappable(folio)) {
        int i;
        for (i = 0; i < nr; i++) {
            struct page *page = folio_page(folio, i);
            /* increment count (starts at -1) */
            atomic_set(&page->_mapcount, 0);
            SetPageAnonExclusive(page);
        }
        atomic_set(&folio->_nr_pages_mapped, nr);
    } else {
        /* increment count (starts at -1) */
        atomic_set(&folio->_entire_mapcount, 0);
        atomic_set(&folio->_nr_pages_mapped, ENTIRELY_MAPPED);
        SetPageAnonExclusive(&folio->page);
        __lruvec_stat_mod_folio(folio, NR_ANON_THPS, nr);
    }
    __lruvec_stat_mod_folio(folio, NR_ANON_MAPPED, nr);
}
```

**最终在 folio_add_new_anon_rma 中调用__folio_set_anon 方法建立 struct page 与 struct anon_vma** 之间的关系。

```
static void __folio_set_anon(struct folio *folio, struct vm_area_struct
*vma,
                unsigned long address, bool exclusive)
{
    struct anon_vma *anon_vma = vma->anon_vma;
    BUG_ON(!anon_vma);
    if (!exclusive)
        anon_vma = anon_vma->root;
    //低位置 1
    anon_vma = (void *) anon_vma + PAGE_MAPPING_ANON;
    // 转换为 address_space 指针赋值给 page 结构中的 mapping 字段
    WRITE_ONCE(folio->mapping, (struct address_space *) anon_vma);
```

```
    // page 结构中的 index 表示该匿名页在虚拟内存区域 vma 中的偏移
    folio->index = linear_page_index(vma, address);
}
```

PAGE_MAPPING_ANON 常量定义在内核 **/include/linux/page-flags.h** 文件中：

```
#define PAGE_MAPPING_ANON   0x1
```

如果当前物理内存页 struct page 是一个匿名页的话，那么 mapping 指针的最低位会被设置为 1 ，指向该匿名页在进程虚拟内存空间中的匿名映射区域 struct anon_vma 结构（每个匿名页对应唯一的 anon_vma 结构），用于物理内存到虚拟内存的反向映射。

anon_vma 指针加上 PAGE_MAPPING_ANON ，并转换为 address_space 指针，这样可确保 address_space 指针的低位为 1 表示匿名页。

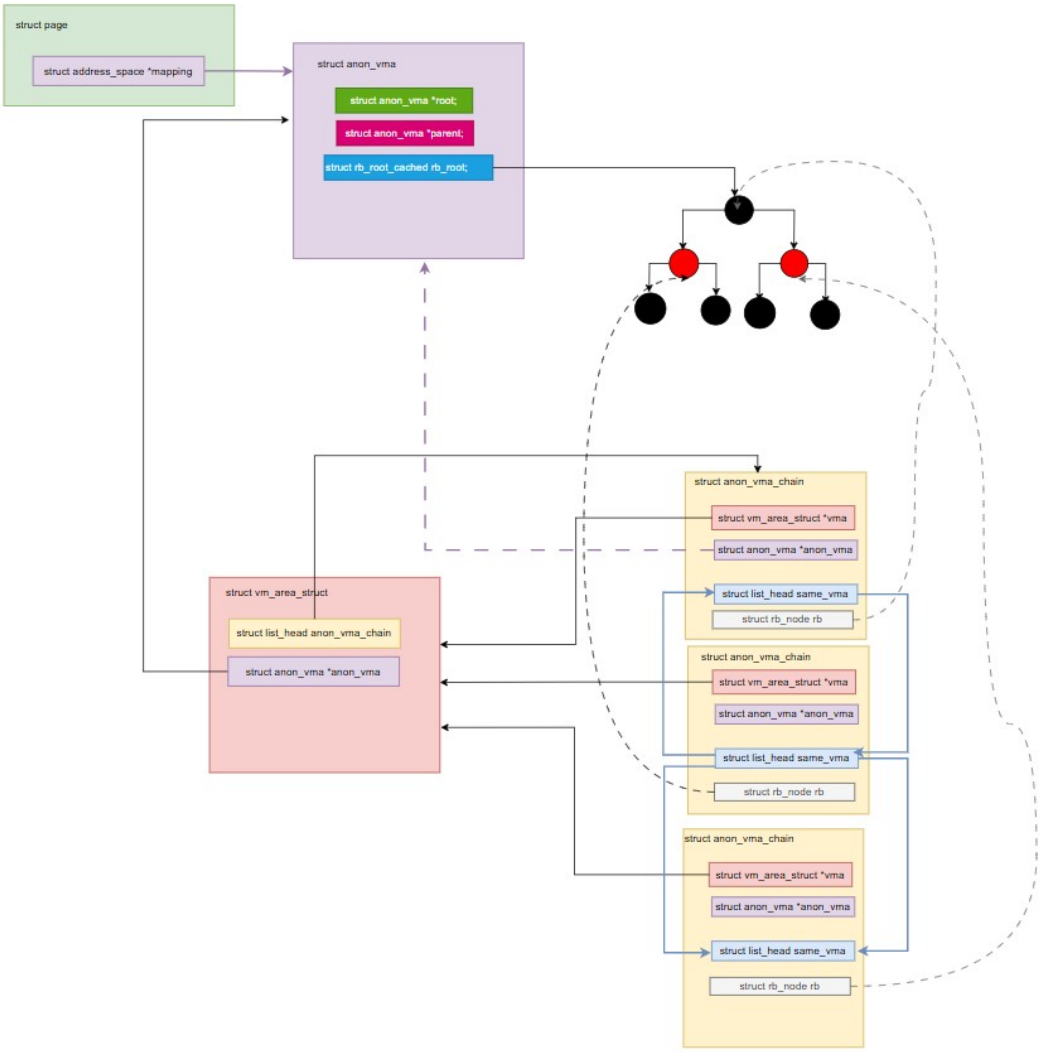
如果当前物理内存页 struct page 是一个文件页的话，那么 mapping 指针的最低位会被设置为 0 ，指向该内存页关联文件的 struct address_space（页高速缓存）。pgoff_t index 字段表示该内存页 page 在页高速缓存中的 index 索引，也表示该内存页中的文件数据在文件内部的偏移 offset。偏移单位为 page size。

对于文件页来说，page 结构的 mapping 指针最低位本来就是 0 ，因为 address_space 类型的指针实现总是对齐至 sizeof(long)，因此在 Linux 支持的所有计算机上，指向 address_space 实例的指针最低位总是为 0 。内核可以通过这个技巧检查 page 结构中的 mapping 指针的最低位来判断该物理内存页到底是匿名页还是文件页。

**文件页的 page 结构的 index** 属性表示该内存页 page 在磁盘文件中的偏移 offset ，偏移单位为 page size 。**匿名页的 page 结构中的 index** 属性表示表示匿名页在对应进程虚拟内存区域 VMA 中的偏移。由 linear_page_index 函数表示:

```
static inline pgoff_t linear_page_index(struct vm_area_struct *vma,
                    unsigned long address)
{
    pgoff_t pgoff;
    pgoff = (address - vma->vm_start) >> PAGE_SHIFT;
    pgoff += vma->vm_pgoff;
    return pgoff;
}

struct wait_page_key {
    struct folio *folio;
    int bit_nr;
    int page_match;
};
```

sruct anon_vma 、 struct anon_vma_chain、 struct page、与 vm_area_struct 整体关系流程图：




## 4.9 内存页回收相关属性

struct page 结构体中与内存页回收相关的属性是 struct list_head lru。物理内存页在内核中分为匿名页和文件页。内核中有两个链表分别为：active 链表和 inactive 链表。
其中， active 链表用来存放访问非常频繁的内存页（热页）， inactive 链表用来存放访问不怎么频繁的内存页（冷页），当内存紧张的时候，内核就会优先将 inactive 链表中的内存页置换出去。

内核在回收内存的时候，这两个列表中的回收优先级为：inactive 链表尾部 > inactive 链表头部 > active 链表尾部 > active 链表头部。内存回收的关键是如何实现一个高效的页面替换算法 PFRA (Page Frame Replacement Algorithm) 。内核中使用最频繁的页面置换算法是 LRU (Least-Recently-Used) 算法。

LRU 算法的核心思想就是那些最近最少使用的页面，在未来的一段时间内可能也不会再次被使用，所以在内存紧张的时候，会优先将这些最近最少使用的页面置换出去。

LRU 算法更多的是在时间维度上的考量，突出最近最少使用，但是它并没有考量到使用频率的影响，假设有这样一种状况，一个页面被疯狂频繁的使用，毫无疑问它肯定是一个

热页，但是这个页面最近的一次访问时间离现在稍微久了一点，此时进来大量的页面，这些页面的特点是只会使用一两次，以后将再也不会用到。

在这种情况下，根据 LRU 的语义这个之前频繁地被疯狂访问的页面就会被置换出去了（本来应该将这些大量一次性访问的页面置换出去的），当这个页面在不久之后要被访问时，此时已经不在内存中了，还需要在重新置换进来，造成性能的损耗。这种现象也叫 Page Thrashing（页面颠簸）。

因此，内核为了将页面使用频率这个重要的考量因素加入进来，于是就引入了 active 链表和 inactive 链表。工作原理如下：

1. 首先 inactive 链表的尾部存放的是访问频率最低并且最少访问的页面，在内存紧张的时候，这些页面被置换出去的优先级是最大的。

2. 对于文件页来说，当它被第一次读取的时候，内核会将它放置在 inactive 链表的头部，如果它继续被访问，则会提升至 active 链表的尾部。如果它没有继续被访问，则会随着新文件页的进入，内核会将它慢慢的推到 inactive 链表的尾部，如果此时再次被访问则会直接被提升到 active 链表的头部。此时页面的使用频率这个因素已经被考量了进来。

3. 对于匿名页来说，当它被第一次读取的时候，内核会直接将它放置在 active 链表的尾部，注意不是 inactive 链表的头部，这里和文件页不同。因为匿名页的换出 Swap Out 成本会更大，内核会对匿名页更加优待。当匿名页再次被访问的时候就会被被提升到 active 链表的头部。

4. 当遇到内存紧张的情况需要换页时，内核会从 active 链表的尾部开始扫描，将一定量的页面降级到 inactive 链表头部，这样一来原来位于 inactive 链表尾部的页面就会被置换出去。

内核在回收内存的时候，这两个列表中的回收优先级为：inactive 链表尾部 > inactive 链表头部 > active 链表尾部 > active 链表头部。

内核参数 swappiness，为 0 到 100，默认为 60。用于表示 Swap 机制的积极程度，数值越大，Swap 的积极程度，越高越倾向于回收匿名页。数值越小，Swap 的积极程度越低，越倾向于回收文件页。

回收匿名页和回收文件页的代价是不一样的，回收匿名页代价会更高一点，所以引入 swappiness 来控制内核回收的倾向。swappiness 只是表示 Swap 积极的程度，当内存非常紧张的时候，即使将 swappiness 设置为 0 ，也还是会发生 Swap 的。

假设只有 active 链表和 inactive 链表，不对这两个链表进行匿名页和文件页的归类，在需要页面置换的时候，内核会先从 active 链表尾部开始扫描，当 swappiness 被设置为 0 时，内核只会置换文件页，不会置换匿名页。

由于 active 链表和 inactive 链表没有进行物理页面类型的归类，所以链表中既会有匿名页也会有文件页，如果链表中有大量的匿名页的话，内核就会不断的跳过这些匿名页去寻找文件页，并将文件页替换出去，这样从性能上来说肯定是低效的。

因此内核将 active 链表和 inactive 链表按照匿名页和文件页进行了归类，当 swappiness 被设置为 0 时，内核只需要去 nr_zone_active_file 和 nr_zone_inactive_file 链表中扫描即可，提升了性能。

除了以上介绍的四种 LRU 链表（匿名页的 active 链表，inactive 链表和文件页的 active 链表， inactive 链表）之外，内核还有一种链表，比如进程可以通过 mlock() 等系统调用把内存页锁定在内存里，保证该内存页无论如何不会被置换出去，比如出于安全或者性能的考虑，页面中可能会包含一些敏感的信息不想被 swap 到磁盘上导致泄密，或者一些频繁访问的内存页必须一直贮存在内存中。

当这些被锁定在内存中的页面很多时，内核在扫描 active 链表的时候也不得不跳过这

些页面，所以内核又将这些被锁定的页面单独拎出来放在一个独立的链表中。

内核会根据不同的情况将一个物理页放置在这五种链表其中一个上。物理页的 struct page 结构中的 struct list_head lru 属性就是用来标识该物理页究竟被内核放置在哪个链表上。

```
struct page {
            ......
            union {
                struct list_head lru;
            }
            ......
}
```

## 4.10 物理内存页属性和状态的标志位 flag

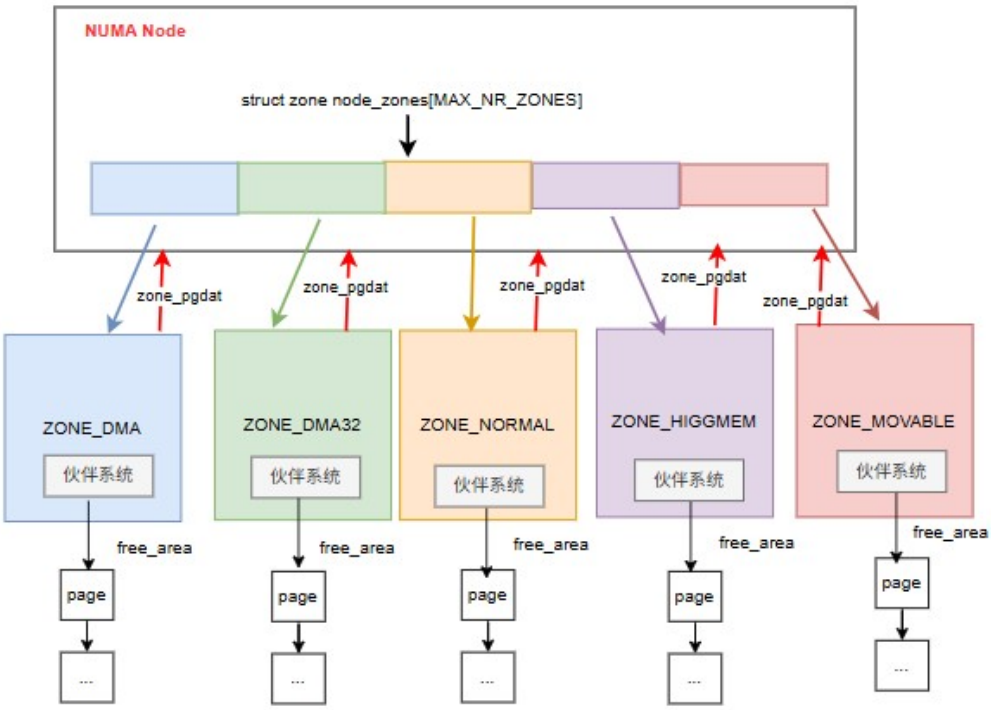
问题：

1、在稀疏内存模型中内核如何确定某个物理内存页到底属于哪个 mem_section？

2、在 NUMA 架构下，内核如何确定某个物理内存页究竟属于哪个 NUMA 节点，以及属于哪块内存区域 zone ？

内核为了能够更灵活地管理粒度更小的连续物理内存，于是就此引入了 SPARSEMEM 稀疏内存模型。SPARSEMEM 稀疏内存模型的核心思想就是提供对粒度更小的连续内存块进行精细的管理，用于管理连续内存块的单元被称作 section 。内核中用于描述 section 的数据结构是 struct mem_section。由于 section 被用作管理小粒度的连续内存块，这些小的连续物理内存在 section 中也是通过数组的方式被组织管理。每个 struct mem_section 结构体中有一个 section_mem_map 指针用于指向连续内存的 page 数组。而所有的 mem_section 也会被存放在一个全局的数组 mem_section 中。

在稀疏内存模型中内核如何确定某个物理内存页到底属于哪个 mem_section？
物理内存在内核中管理的层级关系为：None -> Zone -> page。三级结构。




struct page 结构中的 flags 包含了定位物理内存页属性和标志位的相关信息。unsigned long 类型的 flags 字段中，高 8 位用来表示 struct page 的定位信息，剩余低位表示特定

的标志位。

struct page 与其所属上层结构转换的相应函数定义在 **/include/linux/mm.h** 文件中：

```
static inline unsigned long page_to_section(const struct page *page)
{
    return (page->flags >> SECTIONS_PGSHIFT) & SECTIONS_MASK;
}
static inline pg_data_t *page_pgdat(const struct page *page)
{
    return NODE_DATA(page_to_nid(page));
}
static inline struct zone *page_zone(const struct page *page)
{
    return
&NODE_DATA(page_to_nid(page))->node_zones[page_zonenum(page)];
}
```

flags 字段低位比特表示的物理内存页的标志位定义在内核 /include/linux/page-flags.h 文件中：

```
enum pageflags {
    PG_locked,      /* Page is locked. Don't touch. */
    PG_writeback,       /* Page is under writeback */
    PG_referenced,
    PG_uptodate,
    PG_dirty,
    PG_lru,
    PG_head,        /* Must be in bit 6 */
    PG_waiters,     /* Page has waiters, check its waitqueue. Must be bit
#7 and in the same byte as "PG_locked" */
    PG_active,
    PG_workingset,
    PG_error,
    PG_slab,
    PG_owner_priv_1,    /* Owner use. If pagecache, fs may use*/
    PG_arch_1,
    PG_reserved,
```

```
    PG_private,     /* If pagecache, has fs-private data */
    ................

    };
```

**PG_locked** 表示该物理页面已经被锁定，如果该标志位置位，说明有使用者正在操作该 page，则内核的其他部分不允许访问该页， 这可以防止内存管理出现竞态条件，例如：在从硬盘读取数据到 page 时。

**PG_mlocked** 表示该物理内存页被进程通过 mlock 系统调用锁定常驻在内存中，不会被置换出去。

**PG_referenced** 表示该物理页面刚刚被访问过。

**PG_active** 表示该物理页位于 active list 链表中。PG_referenced 和 PG_active 共同控制了系统使用该内存页的活跃程度，在内存回收的时候这两个信息非常重要。

**PG_uptodate** 表示该物理页的数据已经从块设备中读取到内存中，并且期间没有出错。

**PG_readahead** 当进程在顺序访问文件的时候，内核会预读若干相邻的文件页数据到 page 中，物理页 page 结构设置了该标志位，表示它是一个正在被内核预读的页。

**PG_dirty** 物理内存页的脏页标识，表示该物理内存页中的数据已经被进程修改，但还没有同步会磁盘中。

**PG_lru** 表示该物理内存页现在被放置在哪个 lru 链表上，比如：是在 active list 链表中 ？ 还是在 inactive list 链表中 ？

**PG_highmem** 表示该物理内存页是在高端内存中。

**PG_writeback** 表示该物理内存页正在被内核的 pdflush 线程回写到磁盘中。

**PG_slab** 表示该物理内存页属于 slab 分配器所管理的一部分。

**PG_swapcache** 表示该物理内存页处于 swap cache 中。 struct page 中的 private 指针这时指向 swap_entry_t 。

**PG_reclaim** 表示该物理内存页已经被内核选中即将要进行回收。

**PG_buddy** 表示该物理内存页是空闲的并且被伙伴系统所管理。

**PG_compound** 表示物理内存页属于复合页的其中一部分。

**PG_private** 标志被置位的时候表示该 struct page 结构中的 private 指针指向了具体的对象。不同场景指向的对象不同。

内核还定义了一些标准宏，用来检查某个物理内存页 page 是否设置了特定的标志位，以及对这些标志位的操作，这些宏在内核中的实现都是原子的，命名格式如下：

PageXXX(page)：检查 page 是否设置了 PG_XXX 标志位

SetPageXXX(page)：设置 page 的 PG_XXX 标志位

ClearPageXXX(page)：清除 page 的 PG_XXX 标志位

TestSetPageXXX(page)：设置 page 的 PG_XXX 标志位，并返回原值

在很多情况下，内核通常需要等待物理页 page 的某个状态改变，才能继续恢复工作，内核提供了如下两个辅助函数，来实现在特定状态的阻塞等待：

```
static inline void wait_on_page_locked(struct page *page)
void wait_on_page_writeback(struct page *page);
```

当物理页面在锁定的状态下，进程调用了 wait_on_page_locked 函数，进程就会阻塞等待直到页面解锁。当物理页面正在被内核回写到磁盘的过程中，进程调用了 wait_on_page_writeback 函数就会进入阻塞状态直到脏页数据被回写到磁盘之后被唤醒。

## 4.11 复合页 compound_page 相关属性

Linux 管理内存的最小单位是 page，每个 page 描述 4K 大小的物理内存，但在一些对于内存敏感的使用场景中，用户往往期望使用一些巨型大页。巨型大页就是通过两个或者多个物理上连续的内存页 page 组装成的一个比普通内存页 page 更大的页.

使用巨型页的一些优缺点:

1、巨型页要比普通的 4K 内存页要大很多，遇到缺页中断的情况就会相对减少，由于减少了缺页中断所以性能会更高。

2、于巨型页比普通页要大，所以巨型页需要的页表项要比普通页要少，页表项里保存了虚拟内存地址与物理内存地址的映射关系，当 CPU 访问内存的时候需要频繁通过 MMU 访问页表项获取物理内存地址，由于要频繁访问，所以页表项一般会缓存在 TLB 中，因为巨型页需要的页表项较少，所以节约了 TLB 的空间同时降低了 TLB 缓存 MISS 的概率，从而加速了内存访问。

3、当一个内存占用很大的进程通过 fork 系统调用创建子进程的时候，会拷贝父进程的相关资源，其中就包括父进程的页表，由于巨型页使用的页表项少，所以拷贝的时候性能会提升不少。

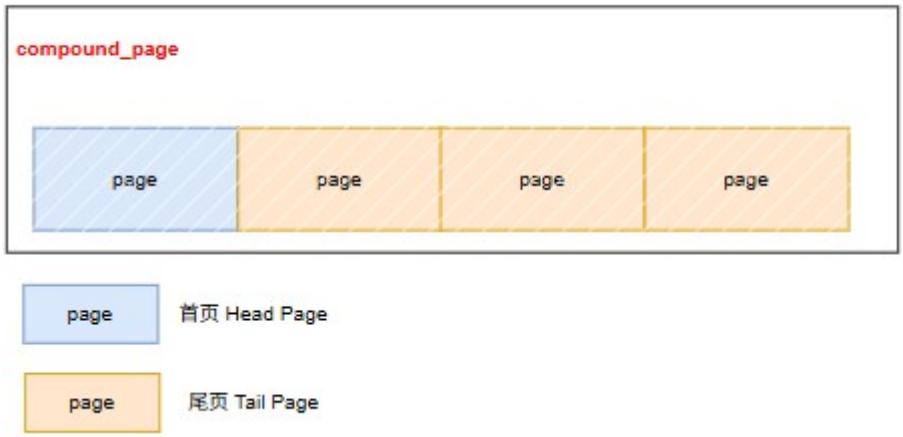
缺点：

1、碎片加剧（物理内存）：巨型页要求物理内存连续，频繁分配 / 释放后，容易产生“无法被巨型页利用的小连续内存块”，加剧物理内存碎片化

2、进程迁移成本高：进程在 NUMA 节点间迁移时，巨型页的连续物理内存需同步迁移（或重新分配），相比 4KB 页，迁移耗时更长，影响调度响应速度；

3、共享与 COW 效率低：写时复制（COW）机制对巨型页不友好 —— 哪怕修改巨型页中的 1 字节，也需复制整个巨型页（如 2MB），导致内存占用翻倍，且复制过程耗时更长；

巨型页（compound_page）是由多个物理上连续的普通 page 组成的，在内核的视角里它还是被当做一个特殊内存页来看待。例如， 一个由 4 个连续的普通内存页 page 组成的一个 compound_page。




组成复合页的第一个 page 我们称之为首页（Head Page），其余的均称之为尾页

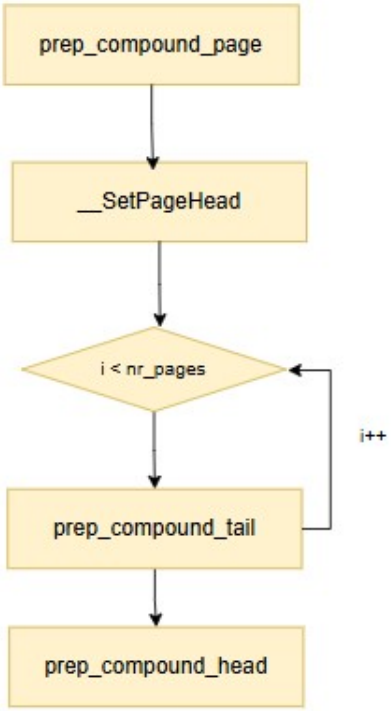
（Tail Page）。

struct page 中关于描述 compound_page 的相关字段：

```
struct page {
        // 首页 page 中的 flags 会被设置为 PG_head 表示复合页的第一页
        unsigned long flags;
        // 其余尾页会通过该字段指向首页,首页不使用该字段
        struct {    /* Tail pages of compound page */
            unsigned long compound_head;    /* Bit zero is set */
        };
}
```

首页对应的 struct page 结构里的 flags 会被设置为 PG_head，表示这是复合页的第一页。

内核中调用函数 prep_compound_page 对符合也进程初始化操作。整体处理逻辑：




```
void prep_compound_page(struct page *page, unsigned int order)
{
    int i;
    //复合页包含的普通页总数（2^order）
    int nr_pages = 1 << order;
    //将复合页中的首页设置为头部页
    __SetPageHead(page);
```

```
    //除第一个页外其余页均初始化为尾页
    for (i = 1; i < nr_pages; i++)
        prep_compound_tail(page, i);
    //完善头部页属性
    prep_compound_head(page, order);
}
```

**prep_compound_tail**: 将复合页中指定索引的普通页（tail_idx 对应的页）标记为 “尾部页”，并与复合页的 “头部页（Head Page）” 建立关联.

```
static inline void prep_compound_tail(struct page *head, int tail_idx)
{
    //定位当前要初始化的尾部页
    struct page *p = head + tail_idx;
    //标记尾部页身份。
    //mapping 字段原本用于存储页的映射关联（匿名页指向 anon_vma，文件页指向
address_space），
    //而尾部页无需独立映射。
    p->mapping = TAIL_MAPPING;
    // 将复合页的尾页（Tail Page）与头部页（Head Page）关联
    set_compound_head(p, head);
    //清空私有数据
    set_page_private(p, 0);
}
```

**set_compound_head**： 设置内存页的 compound_head 属性。将第 0 位 bit 位置为 1.表示此页是尾页。compound_head 的值是头部页的地址 +1。尾页通过 compound_head 属性可以定位到所属的头部页。

```
static __always_inline void set_compound_head(struct page *page, struct
page *head)
{
    /*将 compound_head 的 0 位 bit 置为 1。物理页地址按照 4k 对齐，指针的低 12 位
都是 0*/
    WRITE_ONCE(page->compound_head, (unsigned long)head + 1);
}
```

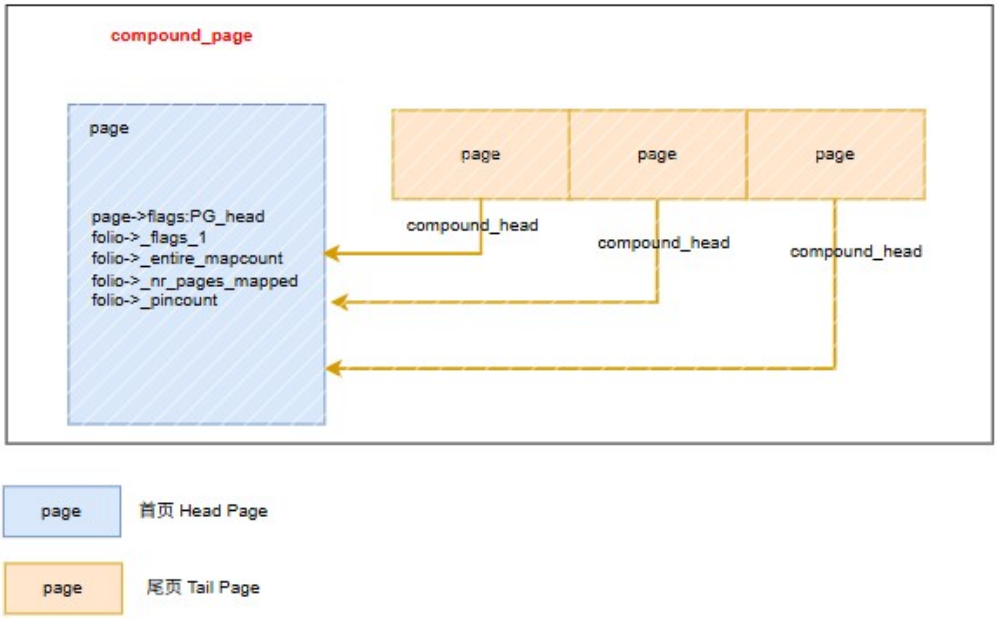
**prep_compound_head :**设置复合页的分配阶数、初始化映射计数、锁定计数。这些属性定义在头部页中。

struct folio 是 Linux 内核基于 struct page 结构体引入的页面组管理结构体。集中管理连续页组的阶数、计数、映射等属性。struct folio 与 struct page 内存布局完全兼容。一个 folio 对应一个 “连续页组”（1 个普通页对应 order=0 folio；512 个 4KB 页组成 2MB 复合页对

应 order=9 folio）。

```
static inline void prep_compound_head(struct page *page, unsigned int order)
{
    struct folio *folio = (struct folio *)page;
    //设置复合页的分配阶数
    folio_set_order(folio, order);
    //初始化整体映射计数。
    /*
    _entire_mapcount 是 folio 的“整体映射计数”，记录复合页被用户态页表映射的次数；
    _entire_mapcount = -1 表示复合页未被拆分，整体映射计数有效
    */
    atomic_set(&folio->_entire_mapcount, -1);
    /*初始化复合已已被用户态映射的页数。复合页刚初始化时，未建立任何用户态映射，因此已映射页数为 0*/
    atomic_set(&folio->_nr_pages_mapped, 0);
    atomic_set(&folio->_pincount, 0);
}
```

复合页中的所有尾页都会通过其对应的 struct folio(page) 结构中的 compound_head 指向首页，这样通过首页和尾页就组装成了一个完整的复合页 compound_page 。




## 4.12 Slab 对象池相关属性

内核中对内存页的分配使用有两种方式：

一种是一页一页的分配使用，这种以页为单位的分配方式内核会向相应内存区域 zone 里的伙伴系统申请以及释放。

另一种方式就是只分配小块的内存，不需要一下分配一页的内存，比如前边章节中提到

的 struct page ，anon_vma_chain ，anon_vma ，vm_area_struct 结构实例的分配，这些结构通常就是几十个字节大小，并不需要按页来分配。

为了满足类似这种小内存分配的需要，Linux 内核使用 slab allocator 分配器来分配，slab 就好比一个对象池，内核中的数据结构对象都对应于一个 slab 对象池，用于分配这些固定类型对象所需要的内存。

它的基本原理是从伙伴系统中申请一整页内存，然后划分成多个大小相等的小块内存被 slab 所管理。这样一来 slab 就和物理内存页 page 发生了关联，由于 slab 管理的单元是物理内存页 page 内进一步划分出来的小块内存，所以当 page 被分配给相应 slab 结构之后，struct page 里也会存放 slab 相关的一些管理数据。

```
struct slab {
    unsigned long __page_flags;

    struct kmem_cache *slab_cache;
    union {
        struct {
            union {
                struct list_head slab_list;
#ifdef CONFIG_SLUB_CPU_PARTIAL
                struct {
                    struct slab *next;
                    int slabs;  /* Nr of slabs left */
                };
#endif
            };
            /* Double-word boundary */
            union {
                struct {
                    void *freelist;     /* first free object */
                    union {
                        unsigned long counters;
                        struct {
                            unsigned inuse:16;
                            unsigned objects:15;
                            unsigned frozen:1;
                        };
                    };
                };
#ifdef system_has_freelist_aba
                freelist_aba_t freelist_counter;
#endif
            };
        };
        struct rcu_head rcu_head;
    };
```

```
    unsigned int __unused;
    atomic_t __page_refcount;
#ifdef CONFIG_MEMCG
    unsigned long memcg_data;
#endif
};
```

**struct list_head slab_list** ：slab 的管理结构中有众多用于管理 page 的链表，比如：完全空闲的 page 链表，完全分配的 page 链表，部分分配的 page 链表，slab_list 用于指定当前 page 位于 slab 中的哪个具体链表上。

**struct slab *next** ： 当 page 位于 slab 结构中的某个管理链表上时，next 指针用于指向链表中的下一个 page。

**int slabs**：表示 slab 中总共拥有的 page 个数。

**unsigned objects** ： slab 中所有的对象个数。

**struct kmem_cache *slab_cache** ： 用于指向当前 page 所属的 slab 管理结构，通过 slab_cache 将 page 和 slab 关联起来。

**void *freelist** ： 指向 page 中的第一个未分配出去的空闲对象，slab 向伙伴系统申请一个或者多个 page，并将一整页 page 划分出多个大小相等的内存块，用于存储特定类型的对象。

**unsigned inuse** ： 表示 slab 中已经被分配出去的对象个数，当该值为 0 时，表示 slab 中所管理的对象全都是空闲的，当所有的空闲对象达到一定数目，该 slab 就会被伙伴系统回收掉。

**unsigned frozen**：当前内存页 page 被 slab 放置在 CPU 本地缓存列表中，frozen = 1，否则 frozen = 0 。